Developing Case-Based Reasoning for Structural Design

Mary Lou Maher and Andrés Gómez de Silva Garza
University of Sydney

CASE-BASED SYSTEMS ENABLE USERS TO RETRIEVE PREVIOUSLY KNOWN DESIGNS FROM MEMORY AND ADAPT THEM TO FIT THE CURRENT DESIGN PROBLEM. THE FOUR CASE-BASED DESIGN SYSTEMS DESCRIBED HERE ILLUSTRATE HOW VARIOUS IMPLEMENTATIONS ACHIEVE DESIGN ASSISTANCE OR DESIGN AUTOMATION OBJECTIVES.

CASE-BASED REASONING IS A problem-solving technique that makes analogies between a problem and previously encountered situations (cases) relevant to solving the problem. Using CBR as a design process model involves the subtasks of recalling previously known designs from memory and adapting these design cases or subcases to fit the current design context.

The detailed development of this process model for a particular design domain proceeds in parallel with the development of the case representation, the case memory organization, and the necessary design knowledge. The selection of an information representation paradigm and the details of its use for a problem-solving domain depend on the intended use of the information, the project information available, and the nature of the domain.

CBR could be used to develop and implement a CBR system. Although that sounds circular, if CBR is a viable approach to problem solving, it can be applied to the development of the reasoning system itself. Toward that end, this article presents four "cases" of case-based building design systems that we've developed at the University of Sydney: CaseCAD, CADsyn, Win, and Demex. These systems exemplify alternative case memory contents and organizations and provide insight into different potential implementations of the recall and adaptation subprocesses.

Implementing case-based structural design

Developing a CBR system starts with the identification of cases that can provide the basis for solving design problems. We began our CBR projects by locating a source of information on structural design projects—a well-known engineering consulting firm that has worked on many medium- to high-rise buildings in Sydney. The firm gave us access to documented project information—primarily design drawings—and the engineers who worked on the projects. Initially, this seemed to be an ideal situation because the engineers were readily available and open to the idea of our developing a case base from their projects.

As it turned out, collecting information about the project designs was much more difficult than collecting the specifications to the building contractors. In the building industry, documentation primarily communicates construction information rather than the designers' intentions about how the design is to behave. Information about the geometric and material properties of the proposed building is available, but explicit functional or behavioral information is not. Therefore, we had to infer design information based on our theoretical understanding of how structural systems behave.

0885-9000/96/$4.00 © 1996 IEEE

Developing Case-Based Reasoning for Structural Design

Mary Lou Maher and Andres Gomez de Silva Garza
University of Sydney

CASE-BASED SYSTEMS ENABLE USERS TO RETRIEVE PREVIOUSLY KNOWN DESIGNS FROM MEMORY AND ADAPT THEM TO FIT THE CURRENT DESIGN PROBLEM. THE FOUR CASE-BASED DESIGN SYSTEMS DESCRIBED HERE ILLUSTRATE HOW VARIOUS IMPLEMENTATIONS ACHIEVE DESIGN ASSISTANCE OR DESIGN AUTOMATION OBJECTIVES.

CASE-BASED REASONING is a problem-solving technique that solves new problems by adapting solutions that were used to solve old problems. Using CBR as a design process model involves the subtasks of recalling previously known designs from memory and adapting these designs or subcases to fit the current design context.

The process model of CBR in design is illustrated in Figure 1. Developing a case-based reasoning system for design involves the development of the case representation, the case memory organization, and the design knowledge required for the selection of an appropriate precedent design and the details of its use for a problem-solving domain depend on the intended use of the information, the project information available, and the nature of the domain.

CBR tools can be used to develop and implement a CBR system that can be used in design, or CBR can be applied to the development of the reasoning system itself. To date, most CBR applications in design have been developed as research prototypes to demonstrate the feasibility of CBR as a design method. In this article, we present four case-based reasoning systems that we have developed in our research laboratory. These systems illustrate various approaches to case representation and organization and provide insight into potential implementations of the recall and adaptation subprocesses.

Implementing case-based structural design

Our interest in case-based reasoning for structural design began with a collaboration with a well-known, innovative consulting firm that has worked on many medium- to high-rise buildings in Sydney. The firm gave us access to documented project information—primarily design drawings—and the engineers who worked on the projects. Initially, this seemed to be a perfect situation because the engineers were already available and open to the idea of developing a case base from their projects.

In fact, collecting information from engineers is not as simple as it seems. Difficulty arises in trying to capture the specifications in the building contractors. In the building industry, documentation primarily communicates the designer's intentions about how the design is to behave. Information about the geometric and material properties can usually be obtained, but explicit functional or behavioral information is not. Therefore, we had to infer design information based on our theoretical understanding of how structural systems behave.

0885-9000/96/$4.00 © 1996 IEEE

We also found that before considering each project, we needed a template outlining the design description. That is, we had to decide ahead of time how to represent the case in memory and then try to match our expectations with the information actually available for a specific design project. Collecting case information is an iterative process that moves from collecting information to changing what we were looking for. So, although we thought that CBR system development starts with identifying cases, we discovered that first we had to determine what information we expected the cases to include and what the case-based reasoner's goal would be. At the same time, we had to allow for revisions of our expectations.

Some commonalities of our four systems reflect our basic approach to CBR. At the most basic level, each system uses attribute-value pairs to represent case memory. The attribute-value pairs are grouped into subsystems and form hierarchies, both in case memory organization and in indexing. The systems group the attribute-value pairs according to the functional, behavioral, and structural roles they play in design reasoning. Although the basic representation paradigm is simple, the organization of cases and case hierarchies builds a more complex representation.

CaseCAD. The CaseCAD project's objective was to propose and implement a design case representation that helps designers find a set of cases relevant to a specified design problem. CaseCAD is a case-browsing tool: it lets users navigate through its memory to look at information that appears relevant to their current interests. It can also retrieve cases autonomously. To pursue the system objectives, we focused on a multimedia case representation to make information readily understood by designers.[1] We also chose flexible memory-indexing techniques that let designers use many different paths to retrieve a case.[2]

Case representation. We collected CaseCAD's design case base for one year, in parallel with system development. We discarded much of the collected data because it was too detailed for solving new conceptual design problems. Finally, we reviewed and collected data from 14 design projects. We implemented four of these projects in the final system to demonstrate recall and adaptation. We did not implement the remaining 10 because they were not necessary to demonstrate these subtasks.

Graphical representations play a vital role in the communication of design information among designers and other building professionals. Graphical media provide a convenient way to handle the complexity of the design description. The human visual system can rapidly capture the wealth of information implicitly and explicitly represented in images, sketches, and drawings. But there are two problems with relying entirely on graphical case representations. First, a graphical representation does not make the design's function or behavior explicit—it addresses only what the design looks like. Second, computers do not reason about complex graphical representations easily. Some case-based design systems, such as Fabel,[3] that use graphical representations for the user interface perform reasoning by converting the images to alternative symbolic representations.

THE ATTRIBUTE CATEGORIZATION PROVIDES AN EFFECTIVE WAY OF REASONING ABOUT THE VARIOUS DESIGN VARIABLES, BECAUSE EACH CLASS OF VARIABLES HAS A DIFFERENT SEMANTICS ASSOCIATED WITH IT.

CaseCAD uses an object-oriented, multimedia case representation. Each object includes a reference to CAD drawings and images, groups of attribute-value pairs, and text-based case descriptions. Cases are decomposed into subcases for easy partial recall and design part reuse. The system uses the attribute-value data for case organization and recall, and the graphics and text descriptions help the designer understand the case information. The attribute-value data also helps designers by summarizing important information in discrete packages and explicitly interpreting the graphical representations.

CaseCAD's object-based representation follows Gero's design prototype framework,[4] which classifies attributes by function, behavior, and structure. CaseCAD adds a fourth category, *relation attributes*, to facilitate implementation of hierarchies and other relational information. Function attributes represent the design's intended purpose—for instance, `support-building-type`, `grid-geometry`, and `resist-wind-load`. Behavior attributes represent the design's response to its environment (its performance); examples are `lateral-displacement`, `total-useable-area`, and `construction-cost`. Structure attributes are the design's physical features, such as `number-of-floors`, `width`, and `height`. Finally, relation attributes represent relationships among design cases, or between design cases and general concepts.

The attribute categorization provides an effective way of reasoning about the various design variables, because each class of variables has a different semantics associated with it. Making the distinctions explicit allows the system to deal with each type differently. Also, meaningful categorization of the vast number of variables that typically describe a building's structural design reduces the representation's complexity.

We implemented case representation in CaseCAD using Framekit, a frame-based representation language derived from Common Lisp. CaseCAD's graphical case representation uses AutoCAD, a general-purpose modeling tool for 3D design visualization, and XFIG, a 2D drawing tool. In CaseCAD, AutoCAD communicates a building design's structural aspects to the user, and XFIG illustrates behavioral information about a case. Figure 1a shows a CaseCAD AutoCAD drawing of a building's structure. Figure 1b shows an XFIG drawing of the same structure's behavior. Figure 1c shows an object-oriented case representation.

In addition to knowledge about specific designs in the form of cases, CaseCAD's memory contains general knowledge about classes of designs, in the form of models. For instance, one model represents CaseCAD's knowledge about medium-rise office buildings. CaseCAD models use object-based representation only, not graphical representation. The object-based model representation reflects the case representation—that is, it consists of sets of attributes in three categories. However, a model's attributes can take values from a finite list or range of values, rather than being associated with one specific value. For instance, the attribute `number-of-floors` has a value of 30 in the `Inn-on-the-Park` case representation, but the

We also found that before considering each project, we needed a template for the project's model. That is, we had to decide ahead of time how to represent the case in memory and then try to match our expectations with the information actually available for a specific design project. Collecting case information is an iterative process that moves from collecting information to changing what we were looking for. So finally, although we thought that CBR systems should reason about cases in terms of the goals and plans of the case's author, we discovered that first we had to determine what information we connected the cases to include and what the case-based reasoner's goal would be. At the same time, we had to allow for revisions of our expectations.

Our two systems also differ in some respects; this reflects our basic approaches to CBR. At the most basic level, each system uses attribute-value pairs to represent case memory. The systems group the attribute-value pairs according to the functional, behavioral, and structural roles they play in design to reason-in a basic way that is simple to implement-about a case's relevance. Although this representation is simple, organizing cases and case hierarchies builds a more complex representation.

CaseCAD. CaseCAD's goal is to support conceptual design by helping designers find cases relevant to their design problem. CaseCAD is a case-browsing tool: It lets users navigate through its memory to look at information that appears relevant to the system's current interests. It can also retrieve cases autonomously to pursue the system goals. We chose these two styles of case retrieval to make information readily understood by designers.[1] We also chose flexible memory indexing techniques that let designers use many different paths to retrieve a case.[2]

Case representation. We developed CaseCAD's design cases from a collection of 24 student projects. Before collecting the data, we had a sense of what information would be useful for solving new conceptual design problems. Finally, we reviewed and collected data from 14 design projects. We implemented four of these projects in the final system to demonstrate recall and adaptation. We did not implement the remaining 10 because they were unnecessary to demonstrate these subtasks.

Graphical representations provide a powerful way to communicate design ideas. Graphical media provide a powerful way to handle the complexity of the design description. The human visual system can rapidly capture the wealth of information implicitly and explicitly represented in images, sketches, and drawings. But there are two problems with relying entirely on graphical case representations. First, a graphical representation does not make the design's function or behavior explicit—it addresses only what the design looks like. Second, computers do not reason about com-

THE ATTRIBUTE CATEGORIZATION PROVIDES AN EFFECTIVE WAY OF REASONING ABOUT THE VARIOUS DESIGN VARIABLES, BECAUSE EACH CLASS OF VARIABLES HAS A DIFFERENT SEMANTICS ASSOCIATED WITH IT.

putational cases as humans do. Computers require explicit, symbolic representations. Therefore, to use graphical representations effectively, designers must provide additional information to make the graphics computationally accessible. Although researchers have developed systems that can automatically extract information from graphical representations, these systems are not yet robust enough to be practical.[3] Instead, designers must explicitly label the graphical representations to identify the relevant features. Labeling is a tedious process, and designers will not do it unless they receive some benefit. Thus, we developed CaseCAD to help designers label graphical representations and to use the labels to help them find relevant cases.

CaseCAD's object-based representation follows Gero's design prototype framework, which classifies attributes as function, behavior, and structure. CaseCAD adds a fourth category, *relation attributes*, to facilitate representation of relationships among design components. Relation attributes describe how components are connected and how they relate to the overall design. Each design object has a set of attributes in each of the four categories. Each attribute has a value or a range of values. For example, a design object might have a function attribute called `support-building-type` with a value of `office`. Or it might have a behavior attribute called `resist-wind-load` with a value of `20 lb/sq ft`. Or it might have a structure attribute called `number-of-floors` with a value of `3`. Or it might have a relation attribute called `adjacent-to` with a value of `Bank-of-America`.

CaseCAD's object-based representation is flexible enough to represent a wide variety of design cases. Each design object can have a different set of attributes, and each attribute can have a different value type. Attribute values can be numbers, symbols, or lists. Numbers can be integers or floating-point numbers. Symbols can be used to represent enumerated types, such as `yes` or `no`, or to represent the design's intended purpose—for instance, `support-building-type`, `office-building-type`, or `apartment-building-type`. Lists can be used to represent multiple values for an attribute, such as `(1 2 3)` for `number-of-floors`. Lists can also be used to represent hierarchical information, such as `(1 (2 3))` for `number-of-floors`, which indicates that the design has a total of three floors, two of which are in the same wing.

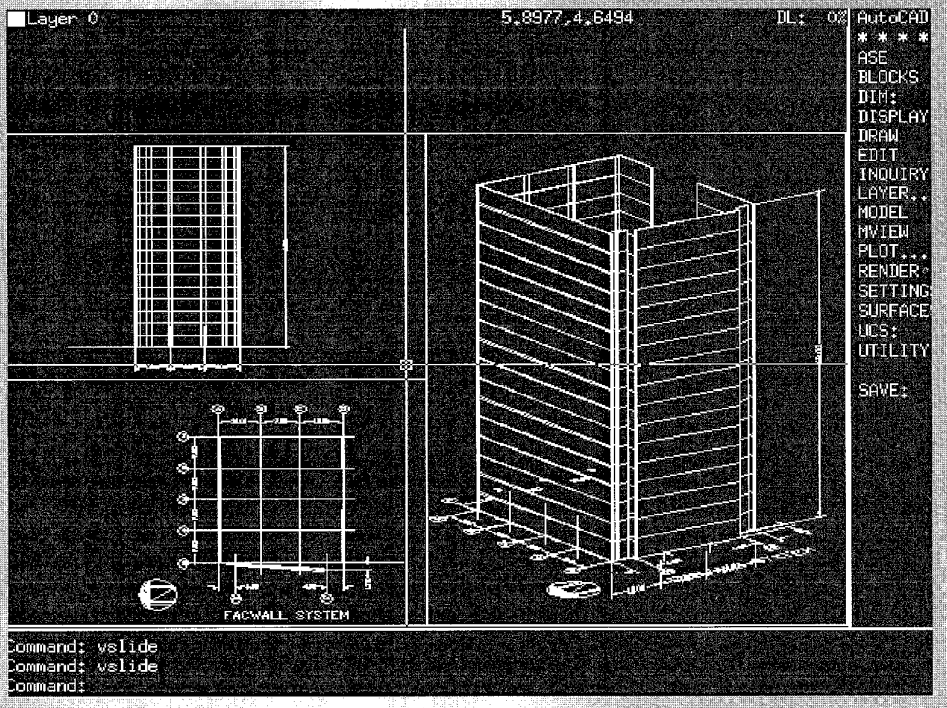
Attributes can also be used to represent relationships among design components. For example, `adjacent-to` `Bank-of-America` indicates that the design object is adjacent to the Bank of America building. Relation attributes can also represent relationships between design cases, or between design cases and general concepts.

Attribute categorization provides an effective way of reasoning about the various design variables, because each class of variables has a different semantics associated with it. Making the distinctions explicit allows the system to deal with each type differently. Also, meaningful categorization of the vast number of variables that typically describe a building's structural design reduces the representation's complexity.

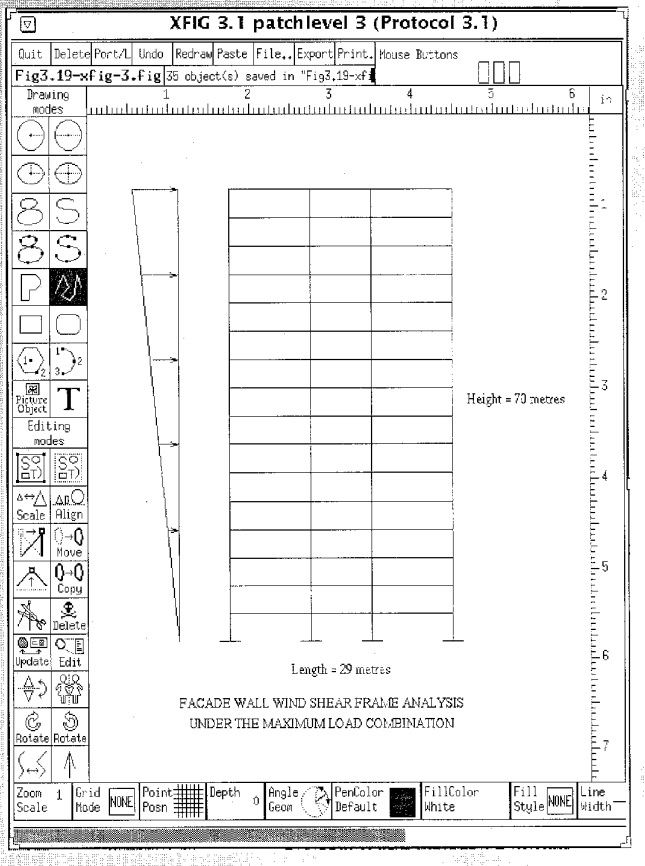
We implemented CaseCAD's representation in CaseCAD using Framekit, a frame-based representation language derived from Common Lisp. CaseCAD's graphical case representation uses AutoCAD, a general-purpose modeling tool for 3D design visualization, and XFIG, a 2D drawing tool. In CaseCAD, AutoCAD creates and displays the graphics, and XFIG prints them. CaseCAD uses AutoCAD's AutoLISP programming language to communicate between Framekit and AutoCAD. Framekit also uses AutoLISP to create XFIG files.

CaseCAD's graphical representation includes references to CAD drawings and images, groups of attribute-value pairs, and text-based case descriptions. Cases are decomposed into subcases for easy partial recall and design part reuse. The system uses the attribute-value data for case organization and recall and the graphics and text for clarification. The attribute-value data also helps designers by summarizing important information in discrete packages and explicitly interpreting the graphical representations

Figure 1. Representations used by CaseCAD: (a) AutoCAD drawing of a building's structure; (b) XFIG drawing of a building's behavior; (c) excerpt of an object-oriented case representation.



(a)

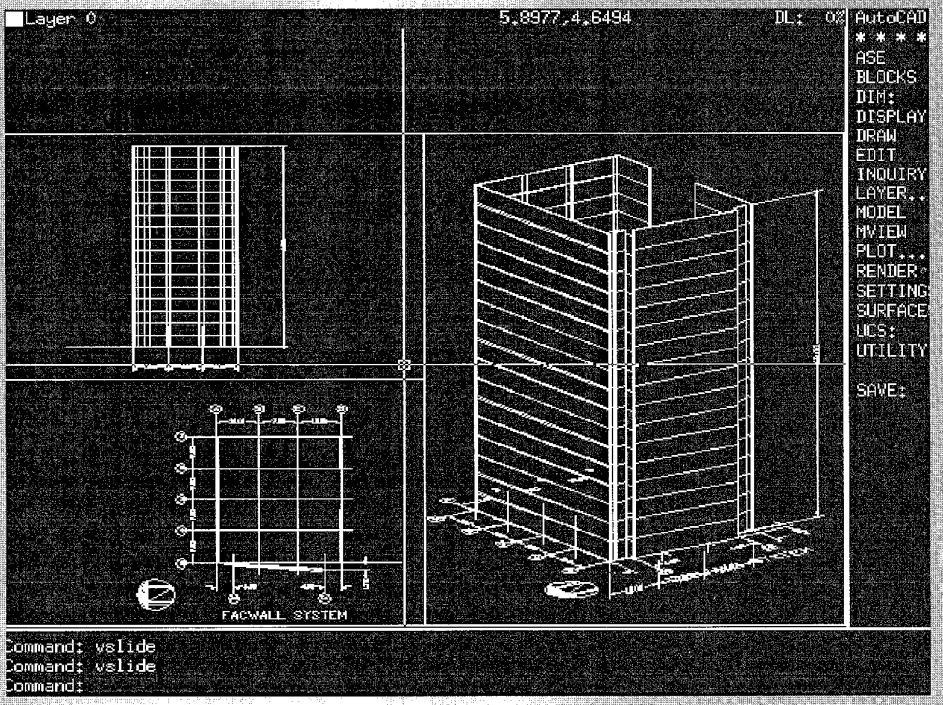


(b)

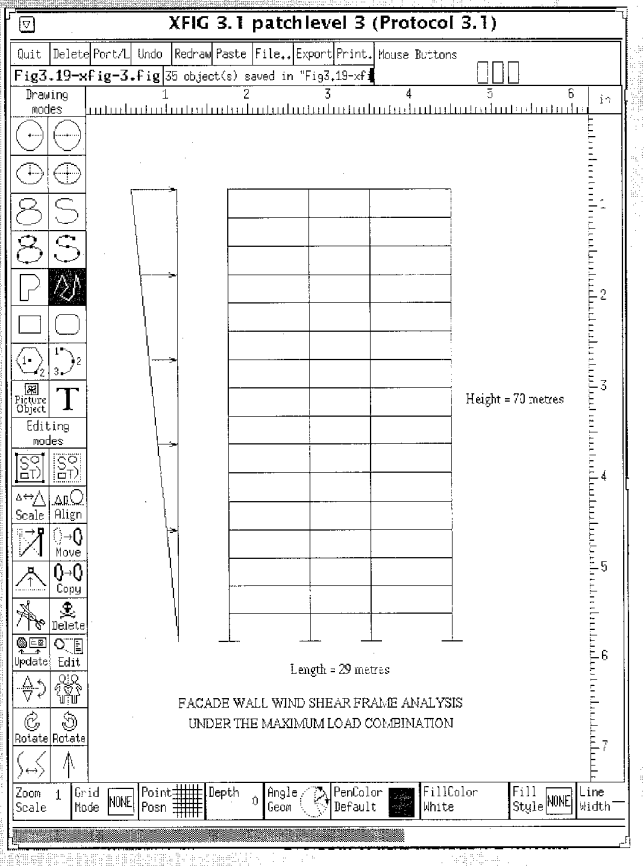
```
SUBCASE1001
DESCRIPTION: The floor system is a com-
bination of standard Rescrete precast
Formplank panels supported by custom-
made precast beam shells and by facade
wall panels. The ends of the Formplank
panels rested on either corbels on the
facade wall units or the upturned edges
of the precast beam shells. These pre-
cast elements were used to produce a
one-way slab, one-way beam system with
structural detailing providing continu-
ity between beams, precast columns, and
facade panels.
RELATION
    part-of: 370Pitt
    instance-of: floor-system
FUNCTION
    transmit-dead-load: 3 kPa
    transmit-live-load: 3 kPa
BEHAVIOR
    fire-resistance-level: 2 hrs
    deflection: 18 mm
    shear-stress: 0.98 Mpa
    xfig-file: "370Pitt/floor-system.fig"
STRUCTURE
    material: precast-concrete
    start-level: 1
    end-level: 17
    span-type:one-way
    maximum-span: 11 m
    typical-span: 9 m
    floor-depth: 250 mm
    span-to-depth-ratio: 44
    maximum-aspect-ratio: 4.2
    acad-file: "370Pitt/floor-system.sld"
```

(c)

Figure 1. Representations used by CaseCAD: (a) AutoCAD drawing of a building's structure; (b) XFIG drawing of a building's behavior; (c) excerpt of an object-oriented case representation.



(a)



(b)

```
SUBCASE1001
DESCRIPTION: The floor system is a com-
bination of standard Rescrete precast
Formplank panels supported by custom-
made precast beam shells and by facade
wall panels. The ends of the Formplank
panels rested on either corbels on the
facade wall units or the upturned edges
of the precast beam shells. These pre-
cast elements were used to produce a
one-way slab, one-way beam system with
structural detailing providing continu-
ity between beams, precast columns, and
facade panels.
RELATION
    part-of: 370Pitt
    instance-of: floor-system
FUNCTION
    transmit-dead-load: 3 kPa
    transmit-live-load: 3 kPa
BEHAVIOR
    fire-resistance-level: 2 hrs
    deflection: 18 mm
    shear-stress: 0.98 Mpa
    xfig-file: "370Pitt/floor-system.fig"
STRUCTURE
    material: precast-concrete
    start-level: 1
    end-level: 17
    span-type: one-way
    maximum-span: 11 m
    typical-span: 9 m
    floor-depth: 250 mm
    span-to-depth-ratio: 44
    maximum-aspect-ratio: 4.2
    acad-file: "370Pitt/floor-system.sld"
```

(c)

```
FLOOR-SYSTEM
RELATION
  part-of: building
  instances: subcase1001, ...
FUNCTION
  transmit-dead-load: 2.9-3.1 kPa
  transmit-live-load: 2.9-3.1 kPa
BEHAVIOR
  deflection: 17-19 mm
  fire-resistance-level: 2
STRUCTURE
  material: precast-concrete, reinforced-concrete
  span-type: one-way
```

Figure 2. Partial CaseCAD design model.

same attribute has a range of 14 to 30 in the medium-rise office building model representation. In the sample subcase in Figure 1c, the relation attribute `instance-of` indicates the model of the general type of structure the subcase belongs to—specifically, `floor-system`. Every case in CaseCAD is an instance of a model known by the system. Figure 2 shows a partial design model.

Browsing and retrieving. CaseCAD's browsing mode enables the user to see the graphical information about a case, using AutoCAD or XFIG, or the object-based representation of the same case. A graphical interface provides menus listing names of cases known to the system, allowing the user to move from one case to another. Figure 3 shows the browser displaying both graphical and text descriptions of the case.

In the retrieving mode, the user gives the system a design problem specification, which CaseCAD uses as an index to retrieve relevant cases from case memory. Design problem specifications, like the object-based case descriptions and models, consist of a set of attribute-value pairs representing the functional, behavioral, and structural requirements of new structural designs. The system measures a model's or a case's relevance to a given problem by finding matches between the attributes and values in the problem specification and those in the object-based description of the memory item.

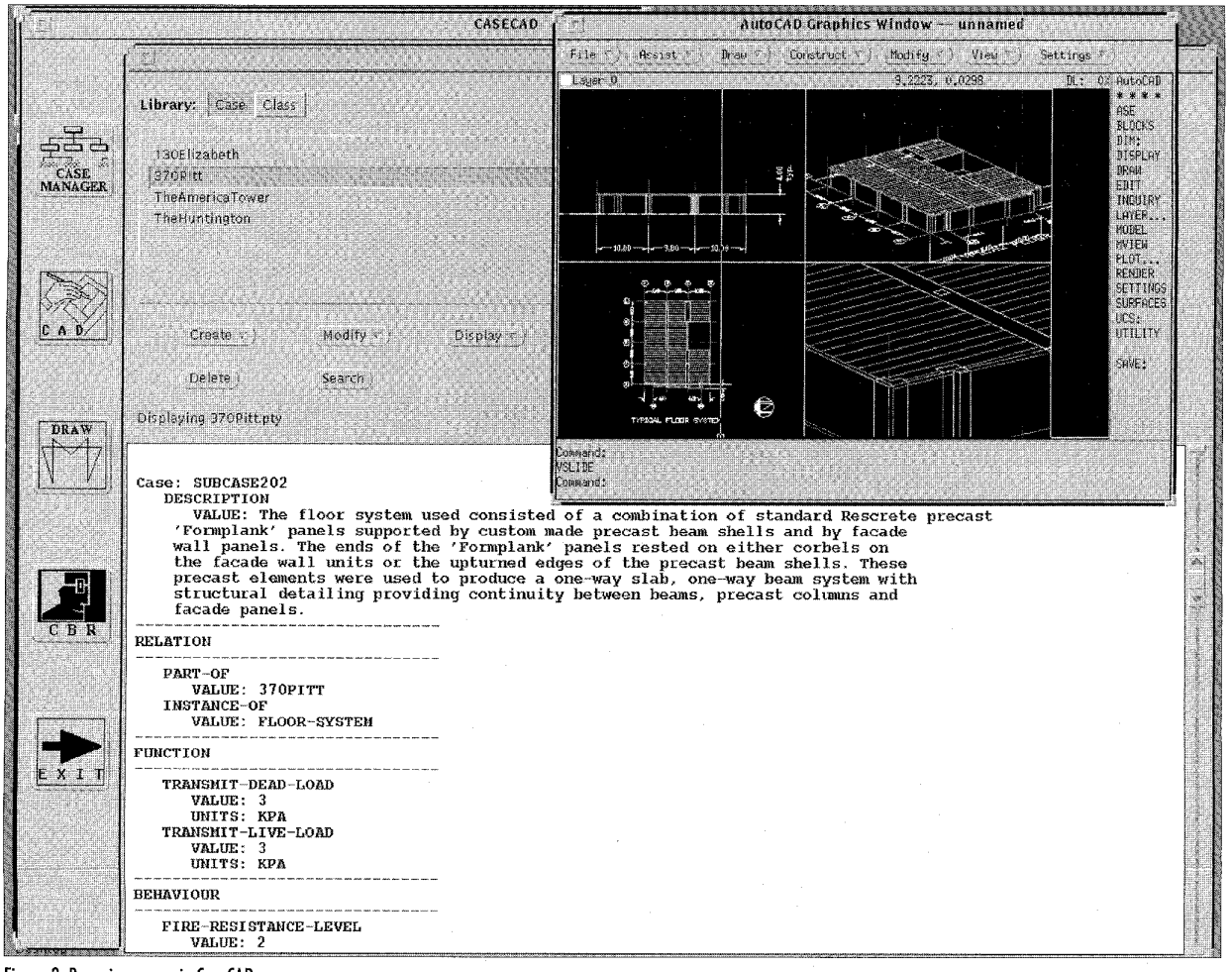


Figure 3. Browsing a case in CaseCAD.

```
FLOOR-SYSTEM
RELATION
 part-of: building
 instances: subcase1001, ...
FUNCTION
 transmit-dead-load: 2.9-3.1 kPa
 transmit-live-load: 2.9-3.1 kPa
BEHAVIOR
 deflection: 17-19 mm
 fire-resistance-level: 2
STRUCTURE
 material: precast-concrete, reinforced-concrete
 span-type: one-way
```

Figure 2. Partial CaseCAD design model.

图2. CaseCAD设计模型的局部视图。

same attribute has a value of 13 to 30. In the example in Figure 2, the relation attribute's instance is `building`, indicating that the model is a part of a building. The structure attribute's instance is `floor-system`, indicating that the model is of the floor system type of structure.

同一属性在中层办公楼模型的表示中取值范围为13至30。在图2所示的实例中，关系属性实例为`building`，表明该模型是建筑物的一部分。结构属性实例为`floor-system`，表明该模型是楼层系统类型的结构。

Browsing and retrieving. CaseCAD's browsing mode enables the user to see the graphical information about a case, using AutoCAD or XFIG, or the object-based representation of the case. CaseCAD's graphical interface provides a menu listing the names of cases known to the system, allowing the user to move from one case to another. Figure 3 shows the browser displaying both graphical and text descriptions of the case.

浏览和检索。 CaseCAD的浏览模式使用户能够以AutoCAD或XFIG格式查看案件的图形信息，或以基于对象的表示形式进行查看。CaseCAD的图形界面提供菜单，列出系统已知的案例名称，使用户能够在不同案例之间进行切换。图3展示了浏览器同时显示该案例的图形描述和文本描述。

In retrieving, CaseCAD gives the user the option of using a design specification, which CaseCAD uses as an index to retrieve relevant cases from case memory. Design problem specifications, like the object-based case descriptions and models, consist of a set of attribute-value pairs representing the functional, behavioral, and structural requirements of new structural designs. The system measures a model's or a case's relevance to a given problem by finding matches between the attributes and values in the problem specification and those in the object-based description of the memory item.

检索。 在检索时，CaseCAD为用户提供了使用设计规范的选项，CaseCAD将其用作索引，从案例库中检索相关案例。设计问题规范与基于对象的案例描述和模型一样，由一组属性—值对构成，用于表征新结构设计的功能、行为及结构需求。系统通过比对问题规范中的属性及其取值与存储项基于对象的描述中的相应属性及其取值之间的匹配程度，来衡量某一模型或案例与给定问题的相关性。

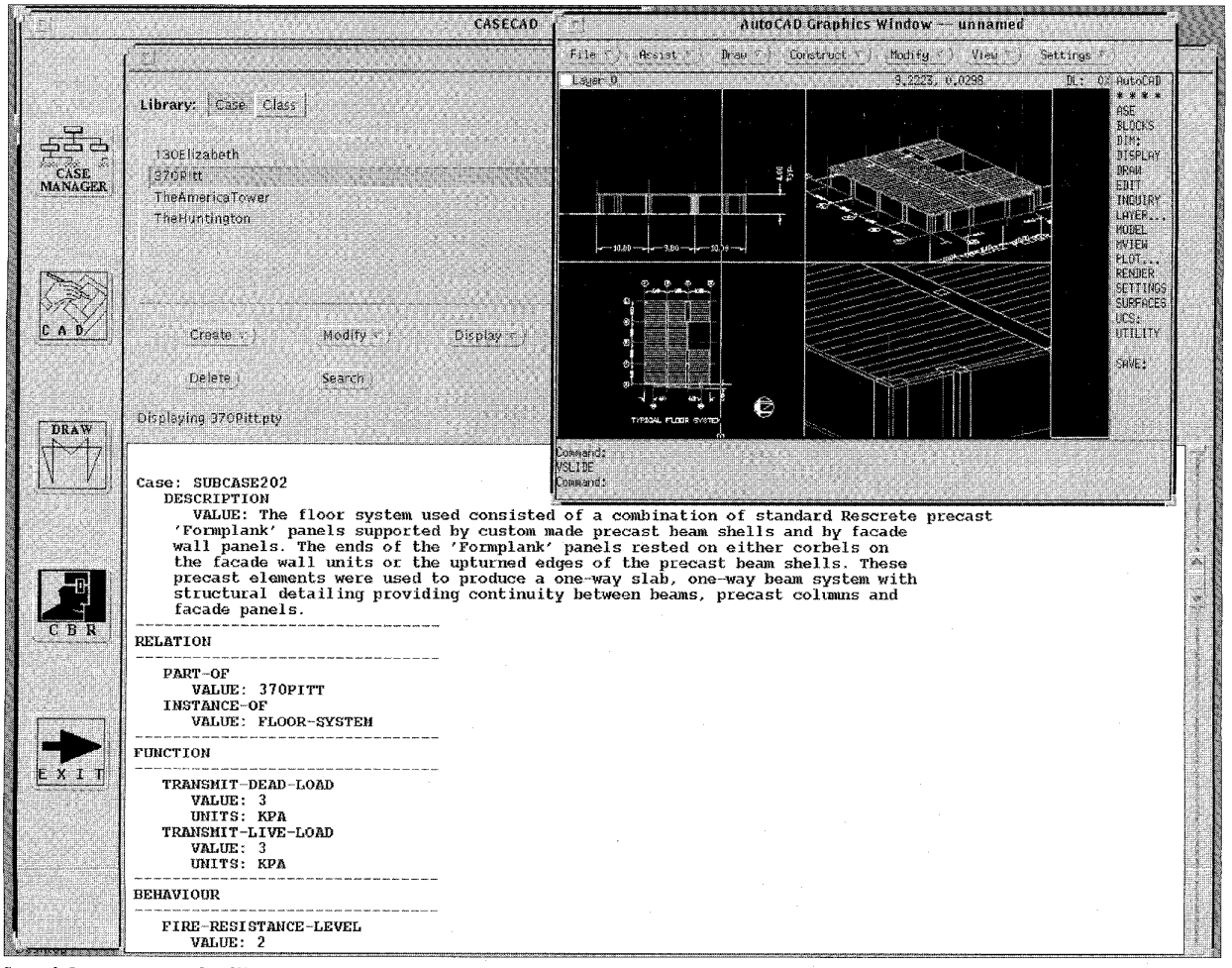


Figure 3. Browsing a case in CaseCAD.

Specifications:

```
FUNCTION
   SUPPORT-GRID-GEOMETRY: (= RECTANGULAR)
   MAXIMUM-SPAN: (= 9 M)
   SUPPORT-BUILDING-TYPE: (= OFFICE)
STRUCTURE
   MATERIAL: (= REINFORCED-CONCRETE)
BEHAVIOR
   COST-OF-CONSTRUCTION: (<= $20 MILLION)
```

Retrieving cases using the FUNCTION specifications:

requirement	no.-of-matchings
SUPPORT-BUILDING-TYPE = OFFICE	7
MAXIMUM-SPAN = 9 M	4
SUPPORT-GRID-GEOMETRY = RECTANGULAR	4

The cases retrieved based on FUNCTION specifications:

subcase	type	no.-of-matchings
130ELIZABETH	MEDIUM-RISE-OFFICE-BUILDING	3
SUBCASE102	RECTANGULAR-GRID-SYSTEM	1
370PITT	MEDIUM-RISE-OFFICE-BUILDING	3
SUBCASE201	RECTANGULAR-GRID-SYSTEM	1
THEAMERICATOWER	HIGH-RISE-OFFICE-BUILDING	3
SUBCASE301	RECTANGULAR-GRID-SYSTEM	1
SUBCASE401	RECTANGULAR-GRID-SYSTEM	1
THEHUNTINGTON	HIGH-RISE-RESIDENTIAL-BUILDING	2

Figure 4. Case retrieval in CaseCAD.

The case that matches the largest number of requirements from the problem specification might not be the best case to use for adaptation. The categorization of attributes into function, behavior, and structure provides a way to focus the retrieval. Rather than using explicit weights to assign importance to each specification, CaseCAD lets the user focus on a relevant set of attributes and ignore others while retrieving cases.

Figure 4 shows an example of a user's problem specification and the system output, detailing cases and subcases that partially match the specification, and the number of requirements they matched. Also shown is the type of structure the case or subcase exemplifies—that is, the model of which the case or subcase is an instance. The retrieval in this example uses only functional attributes as a focus for determining relevance.

Case adaptation. CaseCAD does not perform case adaptation automatically. The designer can modify retrieved cases and use them as new cases in case memory. For the adaptation, the designer uses editing tools to change the object representation, AutoCAD to modify the CAD drawing, and XFIG to modify the images. When the designer adds the case to memory, CaseCAD automatically indexes the object representation, using the attribute-value pairs defined by the designer.

CADsyn. The CADsyn project's objective was to develop a design system that adapts a recalled case automatically to fit a new context.[5] To that end, we developed a case memory representation complementing earlier research on design knowledge representation. The focus on case adaptation was necessary to understand the types and extent of knowledge that would make a CBR system more than a memory-based system. That is, CBR could provide a process model for generating new designs, not just recalling relevant ones.

Case representation. CADsyn's case representation is somewhat similar to CaseCAD's. CADsyn decomposes cases into subcases,

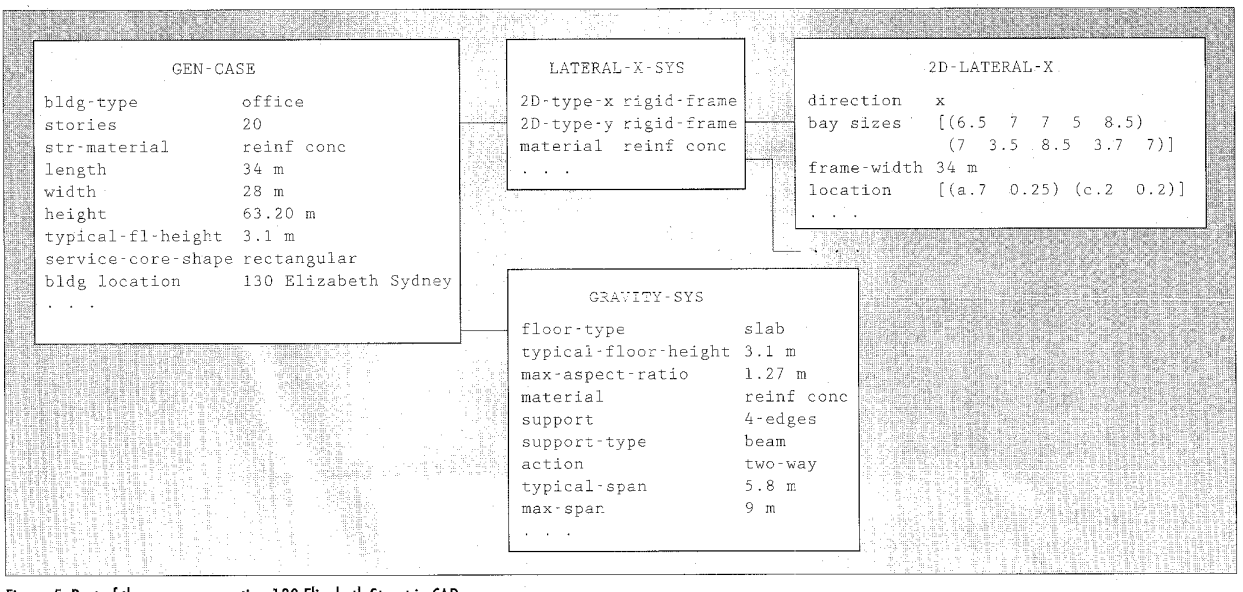


Figure 5. Part of the case representing 130 Elizabeth Street in CADsyn.

Specifications:

```
FUNCTION
   SUPPORT-GRID-GEOMETRY: (= RECTANGULAR)
   MAXIMUM-SPAN: (= 9 M)
   SUPPORT-BUILDING-TYPE: (= OFFICE)
STRUCTURE
   MATERIAL: (= REINFORCED-CONCRETE)
BEHAVIOR
   COST-OF-CONSTRUCTION: (<= $20 MILLION)
```

Retrieving cases using the FUNCTION specifications:

requirement	no.-of-matchings
SUPPORT-BUILDING-TYPE = OFFICE	7
MAXIMUM-SPAN = 9 M	4
SUPPORT-GRID-GEOMETRY = RECTANGULAR	4

The cases retrieved based on FUNCTION specifications:

subcase	type	no.-of-matchings
130ELIZABETH	MEDIUM-RISE-OFFICE-BUILDING	3
SUBCASE102	RECTANGULAR-GRID-SYSTEM	1
370PITT	MEDIUM-RISE-OFFICE-BUILDING	3
SUBCASE201	RECTANGULAR-GRID-SYSTEM	1
THEAMERICATOWER	HIGH-RISE-OFFICE-BUILDING	3
SUBCASE301	RECTANGULAR-GRID-SYSTEM	1
SUBCASE401	RECTANGULAR-GRID-SYSTEM	1
THEHUNTINGTON	HIGH-RISE-RESIDENTIAL-BUILDING	2

Figure 4. Case retrieval in CaseCAD.

图4. 在CaseCAD中的案例检索。

problem might have a large number of requirements, but the best case to use for adaptation might not be the case that matches most of the requirements. Instead, it might be easier to adapt a case that matches fewer requirements.

问题规范中最多数量的需求相匹配的案例，未必是最适于进行适应性调整的案例。特征性划分为功能、行为和结构，为检索提供了聚焦的途径。与通过显式权重来赋予各项规范以重要性不同，CaseCAD允许用户

to each specification, CaseCAD lets the user

在案例检索过程中，重点关注一组相关属性，而忽略其他属性。

Figure 4 shows an example problem statement and the system's output. The output lists the cases and subcases that partially match the specification, and the number of requirements they matched. Also shown is the type of structure the case or subcase

图4显示了一个用户问题规格说明及其系统输出的示例。输出列出了与该规格说明部分匹配的案例及子案例，并标明了它们所匹配的需求数量。此外，图中还显示了该案例或子案例的结构类型。

example uses only functional attributes as a focus for determining relevance.

在本例中，检索仅以功能属性作为确定相关性的依据。

Case adaptation. CaseCAD does not automatically perform case adaptation. The designer can modify retrieved cases and use them as new cases in case memory. For the adaptation, the designer uses editing tools to change the object representation. AutoCAD is used to modify the CAD drawing, and XFIG to modify the image. When the designer adds the case to memory, CaseCAD automatically updates the case representation, using the attribute-value pairs defined by the designer.

案例适配。 CaseCAD不会自动执行案例适配。设计者可以对检索到的案例进行修改，并将其作为新案例存入案例库。在适配过程中，设计者利用编辑工具修改对象表示。借助AutoCAD修改CAD图纸，并使用XFIG修改图像。当设计者将该案例添加到案例库时，CaseCAD会根据设计者定义的属性-值对自动对该案例表示进行更新。

CADsyn project. The objective of the CADsyn project is to develop a system that can automatically adapt retrieved cases to fit a new context. To that end, we developed a case memory representation complementing earlier research on design knowledge representation. The focus on case adaptation was necessary to enable CADsyn to go beyond case-based systems that can only retrieve cases that match most of the requirements. In other words, CBR systems could provide a process model for generating new designs, not just recalling relevant ones.

CADsyn项目的目标是开发一种设计系统，能够自动将已检索的案例适配到新的情境中。为此，我们构建了一种案例记忆表示方法，以补充先前关于设计知识表示的研究成果。聚焦于案例适配这一环节，有助于厘清使基于案例推理（CBR）系统超越单纯记忆系统的知识类型及其范围。换言之，CBR不仅能够检索相关案例，还能够提供用于生成新设计方案的过程模型。

Case representation. CADsyn's case representation scheme is similar to CaseCAD's. CADsyn decomposes cases into subcases,

案例表示。 CADsyn的案例表示方式与CaseCAD的在一定程度上相似。CADsyn将案例分解为子案例，

```
GEN-CASE
bldg-type          office
stories            20
str-material       reinf conc
length             34 m
width              28 m
height             63.20 m
typical-fl-height  3.1 m
service-core-shape rectangular
bldg location      130 Elizabeth Sydney
. . .

LATERAL-X-SYS
2D-type-x rigid-frame
2D-type-y rigid-frame
material  reinf conc
. . .

2D-LATERAL-X
direction   x
bay sizes   [(6.5  7  7  5  8.5)
             (7  3.5  8.5  3.7  7)]
frame-width 34 m
location    [(a.7  0.25) (c.2  0.2)]
. . .

GRAVITY-SYS
floor-type          slab
typical-floor-height 3.1 m
max-aspect-ratio    1.27 m
material            reinf conc
support             4-edges
support-type        beam
action              two-way
typical-span        5.8 m
max-span            9 m
. . .
```

Figure 5. Part of the case representing 130 Elizabeth Street in CADsyn.

```
CONSTRAINT-1
   (bldg-type) in (office parking institution)
   (gravity-system/floor-type) in (flat-plate flat-slab)
CONSTRAINT-2
   (bldg-type) in (hotel apartment)
   (gravity-system/floor-type) = slab
CONSTRAINT-3
   (gravity-system/floor-type) in (flat-plate flat-slab)
   (gravity-system/action) = one-way
   (gravity-system/typical-span) < 10 m
CONSTRAINT-4
   (gravity-system/floor-type) in (flat-plate flat-slab slab)
   (gravity-system/typical-span) > 9 m
   (gravity-system/action) = two-way
CONSTRAINT-5
   (gravity-system/support) in (4-edges cols-only)
   (gravity-system/max-aspect-ratio) > 1.5
   (gravity-system/action) = two-way
```

Figure 6. CADsyn's constraint representation.

and the primary object-based representation uses attribute-value pairs to describe a case. There are major differences, however. First, because CADsyn focuses on case adaptation rather than browsing and retrieval, it does not use graphical, textual, or other human-amenable representations. Adapting the attribute-value pair representation is possible with knowledge-based techniques, but adapting graphical representations or text descriptions is difficult. Second, CADsyn cases are related to models, but we refer to the models as systems and subsystems. This is due to CADsyn's association with EDEsyn,[6] a representation of design synthesis knowledge using decomposition hierarchies called systems. Finally, CADsyn augments the case and system knowledge representation with constraints to adapt cases using a constraint satisfaction approach.

Most cases in CADsyn's memory are the same as in CaseCAD's. CADsyn's memory contains four cases, each with 10 to 14 subcases. The differences between CaseCAD's and CADsyn's case representations lie in the attribute-value pairs and the decomposition into subcases. In CADsyn, we chose attributes and their values according to their utility in implementing the case adaptation subtask and their involvement in constraints. Figure 5 shows an example of a case and a subset of its subcases.

Case retrieval. Design problem specification in CADsyn is the same as in CaseCAD: a set of requirements, in the form of attribute-value pairs, that a design should satisfy to be a valid solution to the problem. The system matches the problem specification with the case descriptions in its memory and retrieves cases and subcases with a partial match. It ranks the retrieved cases according to the number of matching features, favoring the closest match.

CADsyn uses two case-indexing structures: a requirements hierarchy and a subsystem list. The requirements hierarchy uses a conceptual clustering approach.[7] The system traverses the hierarchy according to the problem specification until it finds a list of matching cases. In other words, CADsyn does not search the case memory to match and retrieve cases, but rather searches a separate representation of the requirements hierarchy, which points to relevant cases. The subsystem list associates a subsystem label with a set of subcases that use that label. If CADsyn does not find a relevant case in the requirements hierarchy, it sends the problem specifications to the decomposition reasoner, which identifies a relevant set of subsystems. CADsyn then uses these subsystems to retrieve associated subcases and performs adaptation on the subcases rather than on an entire building design.

Design case adaptation. CADsyn uses constraint satisfaction to adapt design cases, representing constraints as a set of attribute-value pairs that identify an invalid design. If the constraints represented valid assignments of values to variables, they would only help verify the correctness of proposed designs. Using them to represent invalid value assignments helps detect exactly where problems lie in a proposed design, thus focusing attention on design aspects that need alteration to achieve a goal.

Figure 6 shows examples of constraints in CADsyn. `CONSTRAINT-2` can be read as "If the design uses a `floor-type = slab` and `bldg-type = hotel`, then the design is invalid." The constraints are represented as logical relationships among attribute-value pairs, rather than as numerical relationships.

CADsyn's constraint satisfaction modifies a proposed complete design description, based on a retrieved case or subcase, to satisfy the design specifications. Modifying the retrieved case or subcase to match the design specifications typically introduces constraint violations that must be resolved. For example, if a problem specification for a hotel retrieves an office building case, the system must change the value of the attribute `bldg-type` of the retrieved case to `hotel` as the first step in adapting the case to the user's requirements. Additionally, let's assume that the `floor-type` of the retrieved case has a value of `slab` and that the problem specification did not include any reference to the floor type the hotel should have. The value of the attribute `floor-type` in the partially constructed solution would now violate `CONSTRAINT-2`.

Thus, CADsyn's design case adaptation takes an initially inconsistent assignment of values for design attributes and repairs constraint violations through a search process until it achieves a consistent assignment. Reflecting the proposed design's hierarchical representation as a composition of subcases, the constraint satisfaction adaptation generates a feasible design by modifying invalid subsystems (subcases) in the inconsistent potential design. CADsyn detects invalid subsystems by testing constraints and modifies them using its general knowledge of the valid values of design attributes. Because the constraints are not numerical, the system must validate subsystems incrementally, and determining which invalid subsystem to modify first is a central task.

Figure 7 illustrates the subtasks constituting CADsyn's computational model of design case adaptation as constraint satisfaction. These subtasks iterate until they reach a consistent solution to the user's design problem. The figure also shows the knowledge supporting each task. The constraints provide the knowledge necessary to check constraints, find invalid systems, and select a feasible combination of attribute values for a subsystem design. The feasible combination is the combination that violates the fewest constraints. The system uses decomposition knowledge to find invalid systems, to find valid combinations for new design descriptions, and to propagate the effects of the new subsystem design. It uses heuristic rules to select a subsystem to modify. Heuristic rules are necessary because

```
CONSTRAINT-1
   (bldg-type) in (office parking institution)
   (gravity-system/floor-type) in (flat-plate flat-slab)
CONSTRAINT-2
   (bldg-type) in (hotel apartment)
   (gravity-system/floor-type) = slab
CONSTRAINT-3
   (gravity-system/floor-type) in (flat-plate flat-slab)
   (gravity-system/action) = one-way
   (gravity-system/typical-span) < 10 m
CONSTRAINT-4
   (gravity-system/floor-type) in (flat-plate flat-slab slab)
   (gravity-system/typical-span) > 9 m
   (gravity-system/action) = two-way
CONSTRAINT-5
   (gravity-system/support) in (4-edges cols-only)
   (gravity-system/max-aspect-ratio) > 1.5
   (gravity-system/action) = two-way
```

Figure 6. CADsyn's constraint representation.

而由于对象的主要表示形式采用属性-值对来描述案例。然而，两者之间存在显著差异。首先，由于CADsyn侧重于案例的适应而非浏览与检索，因此不使用图形、文本或其他便于人类理解的表示方式。利用知识工程技术可以对属性-值对表示进行适配，但对图形表示或文本描述的适配则较为困难。其次，虽然CADsyn中的案例与模型相关联，但我们将其称为系统和子系统，这是由于CADcases are related to models, but we refer to the models as systems and subsystems. This is due to CADsyn's association with EDESyn's framework, in which CADsyn is knowledge to decompose hierarchies called systems. Finally, CADsyn augments the case and system knowledge representation with constraints to adapt cases using a constraint satisfaction approach.

CADsyn的存储器包含多达81个CaseCAD的案例和四个CADsyn的存储单元，每个单元可以容纳一个案例。每个案例可以细分为1至14个案例。CaseCAD与CADsyn以及CADsyn案例表示上的差异主要体现在属性-值对以及问题案例的分解方式上。在CADsyn中，我们依据属性及其值在实现案例适配于任务中的效用性以及其在约束条件中的参与程度来选择属性及其取值。图5展示了一个案例及其部分子案例的示例。

Case retrieval. 设计问题规范在CADsyn中与其形式与CaseCAD相同，即一组以属性形式表示的需求，即一个设计方案为该问题的有效解应满足的条件。系统将其记录中的案例描述与问题规范进行匹配，并根据匹配特征的数量对检索到的案例进行排序。优先选择匹配度最高的方案。

CADsyn 采用两种案例索引结构来表示其案例库。需求层次结构采用一种概念聚类方法。系统根据问题规格说明遍历该层次结构，直至找到一组匹配的案例。换言之，CADsyn并非直接在案例库中进行匹配与检索，而是通过一种独特的需求层次结构来表示和查找相关案例，该结构指向相应的案例子系统列表。子系统列表则将一个子系统标签与一个案例列表相关联，该列表指向可能适用的案例。如果CADsyn在其中未能找到相关案例，则会使用该标签的子案例关联起来。如果subsystem list associates a subsystem label with a set of subcases that use that label. If CADsyn does not find a relevant case in the

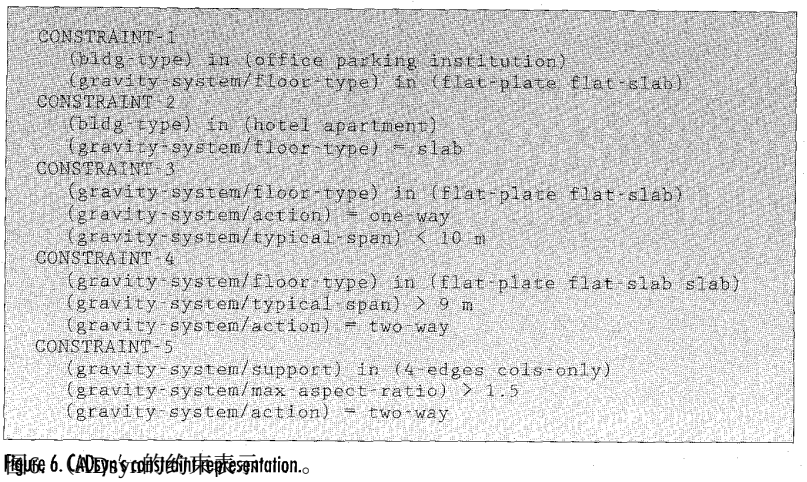


Figure 5. CADsyn's case representation.

在需求层次结构中，它将问题规格作为一组解推理的，后者识别出一组相关的子系统，随后，CADsyn利用这些子系统来检索相应的子案例，并在子案例层面而非整个建筑设计层面进行适配。

Design case adaptation. CADsyn 采用约束满足机制来调整设计实例，将约束表示为一组属性-值对，用于标识无效设计值对，指明哪些取值组合无效。如果这些约束代表的是变量的合法取值，则仅能用于验证所提设计方案的正确性。而将其用于表示非法取值，则有助于精确定位所提设计方案中的问题所在，从而将注意力集中于那些需要修改以达成目标的设计环节。

图6展示了CADsyn中的约束示例。CADsyn CONSTRAINT 2 可解读为：“如果设计采用楼板类型-的极结构且建筑类型为酒店，则该设计无效”。这些约束以属性-值对之间的逻辑关系而非数值关系来表示。

CADsyn 的约束求解机制基于已检索到的案例或子案例，对所提出的完整设计描述进行修改，以满足设计规范。在将检索到的案例或子案例调整至符合设计规范的过程中，通常会引入需要予以解决的约束违反，例如，若针对酒店的设计问题检索到了一栋办公楼的案例，则系统必须首先将该案例中属性 `bldg-type` 的值修改为 `hotel`，从而使其适应用户的需求。此外，假设该检索案例中 `floor-type` 的取值为 `slab`，而问题规格要求采用 `flat-plate` 楼板结构，则在将该案例调整至满足用户需求的过程中，`floor-type` 的取值也应相应调整。

在部分构建的解中，属性 `floor-type` 的取值现将违反约束 `CONSTRAINT-2`。

因此，CADsyn的设计实例适配机制接收一个初始设计属性值集，其中包含对设计属性值的赋值，并通过搜索过程修复约束违反。直至获得一个一致的赋值。鉴于所提出的设计方案以子实例的组合形式进行层次化表示，该约束满足适配机制通过修改不一致的设计值的子实例，来生成一个新的设计。CADsyn通过测试约束来识别无效子系统，并利用其关于设计属性有效取值的通用知识对其进行修正。由于这些约束并非数值型约束，系统必须以增量方式对子系统进行验证，而确定首先修改哪个子系统则是一项启发式任务。

Figure 7 illustrates CADsyn's constraint-based redesign process. CADsyn's control module decomposes the design case adaptation as constraint satisfaction. These subtasks iterate until they reach a consistent solution to the user's design problem. The figure also shows the knowledge supporting each task. The constraints provide the knowledge necessary to check constraint violations and select a feasible combination of attribute values for a subsystem design. The feasible combination is the combination that violates the fewest constraints. The system uses this knowledge to identify invalid subsystems, to find valid combinations for new design descriptions, and to propagate the effects of the new subsystem design. It uses heuristic rules to select a subsystem to modify. Heuristic rules are necessary because

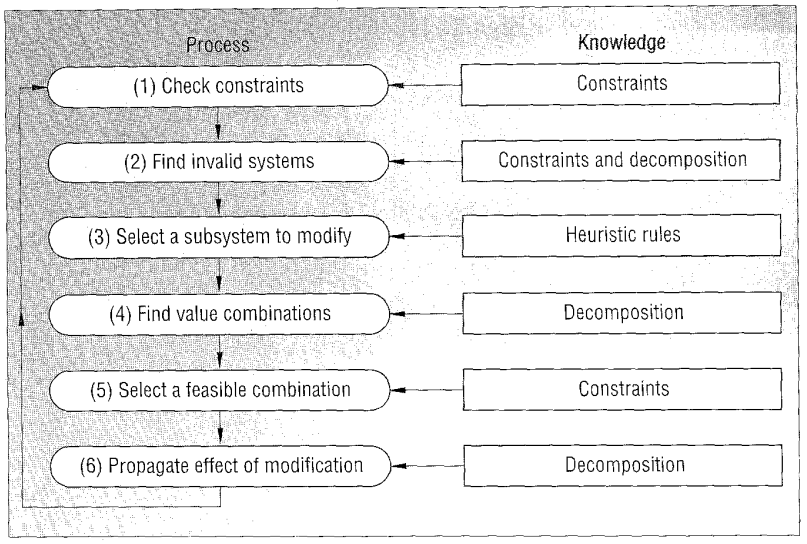


Figure 7. CADsyn's constraint satisfaction process.

the constraints are themselves heuristics, making the design space difficult to analyze.

The complexity of CADsyn's constraint satisfaction process accommodates adaptation of multiple subcases in a hierarchical design representation in which constraints represent logical relationships among design variables. CADsyn's use of constraints contrasts with other systems. For example, Julia, a case-based reasoner for meal planning, uses constraints to adapt cases, representing the meal as a whole case.[8] Cadre, a case-based reasoner for floor plan design, uses constraints to represent numerical relationships among dimension variables.[9]

Win. The Win project's objective was to develop a case-based design assistant with a structural engineering perspective.[10] The project focused on case memory content for a specific structural design aspect: design and layout of wind-resisting systems in medium- and high-rise buildings. The result is a case memory representation specific to that domain.

Case representation. Three of the structural design cases in CaseCAD are represented in the Win system, where each case has approximately four subcases. In Win, cases have more specific information about the engineering performance of lateral load systems than in CaseCAD. Win's case base contains more numerical information about stresses and loads and less high-level symbolic information about the architectural layout. Like CaseCAD, Win's memory contains knowledge in the form of both models and cases, described as attributes and values. Attributes fall into four categories: relation, function, behavior, and structure. Both models and cases are decomposed into subsystems.

Models play different roles in CaseCAD and Win. CaseCAD models are decomposed into subsystems that facilitate the acquisition and retrieval of cases. Win models assist case memory organization and problem specification. Specifically, Win models are defined according to two factors that greatly influence the design of a structure's wind-resisting system: the building's dimensions and the position of the building's core system with respect to the rest of its structure. Thus, Win has four models: medium- and high-rise buildings and central- and offset-core structures. These models' primary purpose is to partition case memory according to the intended use.

Figure 8a shows a model corresponding to the class of medium-rise structures with an offset core. Figure 8b shows a Win case representation of the Inn on the Park, a 30-story hotel and office building in Sydney. Figure 8c shows how Win uses models to partition case memory.

Indexing and retrieving. Case retrieval is an interactive, iterative process in Win. First, Win prompts the user for a set of problem requirements. For each of several important attributes (`number-of-floors`, `width`, `length`, `floor-to-floor-height`, and so on), Win asks the user to input its value, if known. The resulting set of attribute-value pairs becomes the problem specification, which Win uses as an index to its memory. Win first retrieves models or submodels that match the problem requirements before retrieving cases or subcases that are instances of the models. After model retrieval, Win considers the difference between the user's problem specifications and the extent and range of values of attributes in the models. Win requests additional information, or clarification, before initiating case retrieval.

This model-based case indexing allows the system to ignore cases and subcases that are completely irrelevant to a problem and efficiently find those that are relevant. Both the model memory and the case memory are flat structures—that is, retrieval proceeds by attempting to match the index with each item in memory successively. The model memory partitions the case memory. The small number of models and cases in Win's memory makes this flat memory organization feasible; more models would make it inefficient. The user always controls the use of Win's model and case bases and the retrieval and selection of cases.

Demex. The Demex (Design by Memory Exploration) project's objective was to consider flexible retrieval and memory exploration.[11] In this project, we developed the strategies of *index elaboration* and *index revision*, which we'll describe in more detail later. With the Demex case memory, a designer can explore and develop an improved design problem specification, rather than develop a design solution. The system reflects real-world design problem solving, in which a designer does not usually start with a well-specified problem and then move toward a solution, but rather the problem requirements evolve as the designer explores alternative solutions.[12]

Case memory representation. Demex uses CaseCAD's design cases and models, partitioning cases and models into subsystems and categorizing attributes as functional, behavioral, structural, and relational. Because of Demex's highly interactive nature, we devised a memory organization that allows efficient and flexible access to cases and subcases.

In Demex, three memories hold conceptually different types of information: an associative memory that relates the function, behavior, and structure categories with design attributes; a model memory; and a case memory. Although the three memories are stored as separate entities in their own data struc-

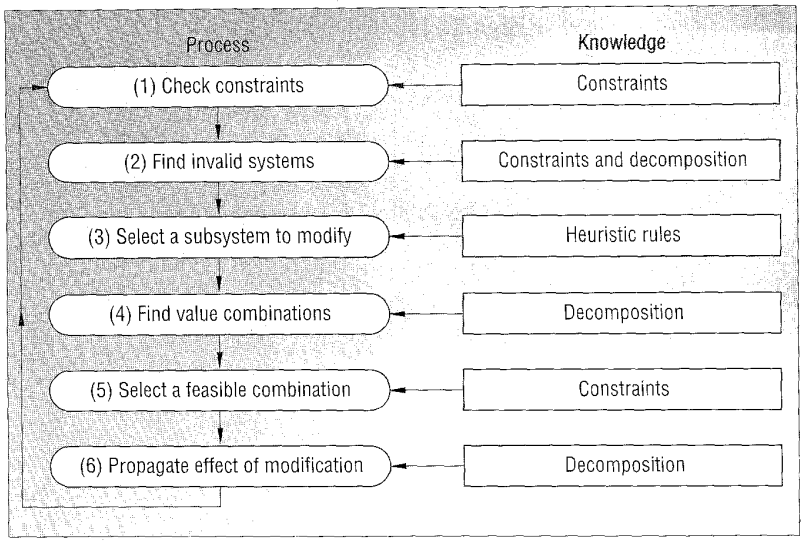


Figure 7. CADsyn's constraint-satisfaction process.

图7. CADsyn的约束满足过程。

这些约束本身即为启发式方法，使得设计空间难以进行分析。

CADsyn的约束满足过程具有高度复杂性，能够适应层次化设计表示中的多个子系统形成鲜明对比。例如，Julia作为一种用于膳食规划的基于案例的推理系统，利用约束来调整案例，将整餐视为一个整体进行处理。而Cadre作为一种用于平面布局设计的基于案例的推理系统，则利用约束来表示部件之间的数值关系。

Win. Win项目的目标是开发一个从结构工程视角出发的基于案例的设计辅助系统。该项目集中于某一特定结构设计领域的案例库内容，即中高层建筑抗风体系的设计与布置，其成果是一种针对该领域特有的案例库表示方法。

Case representation. CaseCAD的三个结构设计案例在Win系统中将以体现，其中每个案例均包含四个子案例。在Win系统中，关于抗侧力体系工程性能的信息比在CaseCAD中更为详尽。Win的案例库包含了更多关于风力与荷载的数值信息，而关于建筑布局的高层次符号化信息则相对较少。与CaseCAD类似，Win的记忆库同样以模型和案例的形式存储知识。

Win首先检索与问题需求相匹配的模型或模型片段，然后将检索作为这些模型实例的案例或子案例。在完成模型检索后，Win会评估用户的问题规格与模型中各属性的取值范围及程度之间的差异，并在启动案例检索之前请求用户提供补充信息或进一步说明。

这种基于模型的案例索引机制使系统能够忽略与问题完全无关的案例或子案例，并高效地检索出相关案例。模型记忆库和案例记忆库均采用扁平化结构——即通过依次将索引与记忆库中的每一项进行匹配来实现检索。模型记忆库对案例记忆库进行划分。由于Win的记忆库中模型和案例的数量较少，这种扁平化的记忆组织方式是可行的。若模型数量增多，则会导致效率下降，届时将需要Win的模型库和案例库的维护工具，以及案例的检索与选择过程的使用，以及案例的检索与选择过程。用户始终掌控Win的模型库和案例库的维护工具，以及案例的检索与选择过程。

Demex. Demex“基于记忆探索的设计”项目的目标是考虑灵活的检索与记忆体系。[11] 在该项目中，我们开发了*index elaboration*和*index revision*两种策略，将在后文予以更详细的阐述。借助Demex，Win like the Demex case memory, a designer's tasks explicit and the recap 的方案。该系统反映了现实中的设计问题求解过程：设计者通常并非从一个已明确界定的问题出发进而寻求解决方案，而是随着对备选方案的探索，问题需求逐步演进。”

Case memory representation. Demex使用CaseCAD的设计案例与模型，将这些案例与模型划分为子系统，并将属性归类为功能属性、行为属性、结构属性和关系属性。基于Demex具有高度交互性的特点，我们设计了一种内存组织方式，以实现对案例及其子案例的高效且灵活的访问。

在Demex系统中，三种记忆分别存储着概念层次结构中关联的信息。每种记忆中的索引将属性（行为属性与结构类别）与属性相关联，而索引记忆则将这些属性与案例记忆中的案例相关联。尽管这三种记忆以不同的实体形式分别存储于各自的数据结构中，

tures to increase efficiency and avoid redundancy, they are interrelated. To begin with, we designed all three using the same attribute-value ontology. Additionally, each model in the model memory contains a pointer to all cases that are instances of that model, and each case in the case memory contains a pointer to the model of which that case is an instance. Cross-indexing of the semantically different knowledge types allows flexible and efficient traversal among them.

Demex's indexing scheme for model and case memory is a tree of attributes and values (see Figure 9). The system can rapidly retrieve relevant models and cases using the corresponding indexing tree to perform a two-step comparison for each attribute-value pair in the problem specification. The first step traverses the tree along a given attribute arc; the second step traverses it along an arc corresponding to a value for that attribute, retrieving a set of cases with the specified attribute-value pair. If only some categories of attributes are of interest, the case retrieval ignores requirements whose attributes belong to an uninteresting category. We designed this three-memory organization and indexing-tree structure especially to support the retrieval strategies we'll describe next.

Index elaboration and revision. Restructuring, reconsidering, and reformulating a problem's specifications are an important aspect of design problem solving. Demex guides the user in reformulating structural design requirements by improving the user's understanding of the problem.

Demex uses two problem reformulation strategies to provide a better understanding of a design problem: model-based index elaboration and case-based index revision. These strategies evolved from discoveries we made in the CaseCAD project about the need for flexible case retrieval.[2]

Making an analogy between the current problem specification and general knowledge in memory, model-based index elaboration retrieves one or more models from memory. The models can provide a better description of the design problem in two ways. First, items retrieved from memory usually contain more information (for example, many more design variables to describe a structure) than does the initial problem specification. The designer can expand the set of specifications with this information.

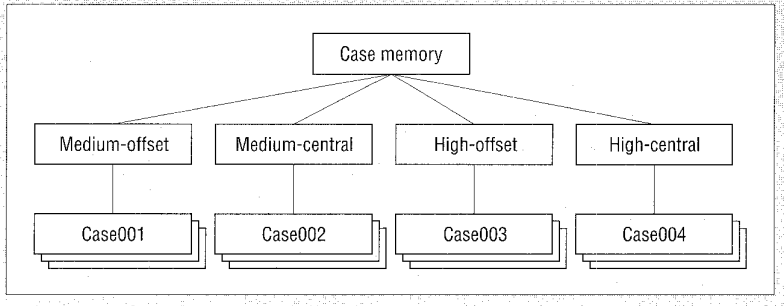
Second, models provide ranges of acceptable values associated with the design

```
***MEDIUM-OFFSET-BUILDING***
FUNCTION
   SUPPORT-BUILDING-TYPE: office, hotel, residential, other
BEHAVIOR
   ABOVE-GROUND-LEVELS: 14-30
   ABOVE-GROUND-HEIGHT: 40-120 m
   CRITICAL-HEIGHT-RATIO: 1-8
   CORSYS-LOCATION-AT-CRITICAL-LEVEL: offset
   CORSYS-AREA-RATIO-AT-CRITICAL-LEVEL: 0.01-0.5
   CORSYS-OFFSET-RATIO-IN-CRITICAL-DIRECTION: 0.1-1
   CORSYS-LENGTH-RATIO-AT-CRITICAL-LEVEL: 0.1-10
   CRITICAL-WIND-DIRECTION: ew, ns
   CRITICAL-WIND-SPEED: 30-60 m/s
STRUCTURE
   OVERALL-LENGTH: 15-120 m
   OVERALL-WIDTH: 15-120 m
   CORSYS-OVERALL-LENGTH: 2.5-25 m
   CORSYS-OVERALL-WIDTH: 2.5-25 m
   CRITICAL-WIND-AREA: 600-14400 m2
```

(a)

```
Case: INN-ON-THE-PARK
DESCRIPTION: The Inn on the Park is a 30-level interna-
tional hotel and commercial office building in the central
business district of Sydney. This development provides 14
levels of accommodation with 250 4-story rooms, occupying
11,700 square meters of floor area; 5 levels of commercial
office space comprising approximately 5,000 square meters
of floor area; 3 levels for hotel facilities; and 7 base-
ment levels for 170 cars. The wind-resisting structure sys-
tem is composed of a core system and a shear wall system.
RELATION
   INSTANCE-OF: MEDIUM-OFFSET-BUILDING
FUNCTION
   SUPPORT-BUILDING-TYPE: hotel
   HOTEL-SPACE (M2): 11700
   HOTEL-ROOMS: 250
   OFFICE-SPACE (M2): 5000
   UNDERGROUND-PARKING-CARS: 170
BEHAVIOR
   STRUCTURE-DESIGN-CODE: AS1480
   WIND-CODE: AS1170
   TOTAL-LEVELS: 30
   TRANSFER-LEVEL: level-8
   ABOVE-GROUND-LEVELS: 23
   UNDER-GROUND-LEVELS: 7
   OVERALL-SHAPE: quadrangular
   TOWER-SHAPE: quadrangular
STRUCTURE
   MATERIAL: insitu-rc
   OVERALL-LENGTH (M): 50
   OVERALL-WIDTH (M): 25
   STD-AREA (M2): 900
   ABOVE-GROUND-HEIGHT (M): 82
   UNDER-GROUND-HEIGHT (M): 20
```

(b)



(c)

Figure 8. Models and cases in Win: (a) design model representation; (b) case representation; (c) using models to partition case memory.

为提高效率并避免冗余，这些要素做出相互关联。首先，我们采用相同的属性设计了这三者。

Moreover, we designed all three using the same attribute-value ontology. In addition, each model in the model memory contains a pointer to the model of which that model is an instance. Cross-indexing of the semantically different knowledge types allows flexible and efficient traversal among them.

Demex用于模型和案例记忆的索引方案是一种由属性及其取值构成的树状结构（见图9）。系统可通过相应的索引树，针对问题规约中的每一对属性-取值，执行两步比较，从而快速检索出相关模型或案例。第一步沿给定属性对应的分支遍历该树；第二步则沿该属性取值所对应的分支进行遍历，以检索出具有指定属性-取值组合的一组案例。若仅关注部分属性类别，则案例检索会忽略那些其属性属于非兴趣类别的约束条件。我们专门设计了这种三重记忆组织及索引树结构，以支持下文将要介绍的检索策略。

The case memory is a tree of attributes and values (see Figure 9). The system can rapidly retrieve relevant models and cases using the corresponding indexing tree to perform a two-step comparison for each attribute-value pair. The first step traverses the node for a given attribute, and the second step traverses it along an arc corresponding to a value for that attribute, retrieving a set of cases with the specified attribute-value pair. If only some categories of case memory are of interest, retrieval ignores cases whose attributes belong to an uninteresting category. We designed this three-memory organization and indexing-tree structure especially to support the retrieval strategies we'll describe next.

Index elaboration and revision. 重构：重新审视并重新表述问题的规格说明是设计问题求解过程中的一个重要环节。Demex通过提示用户对问题的理解，引导用户对结构设计需求进行重新表述。

Index elaboration and revision. Reconstruction—revisiting and restating problem specifications—is an important aspect of design problem solving. Demex guides the user in reformulating structural design requirements by improving the user's understanding of the problem.

Demex采用两种问题重表述策略，以更深地理解设计问题：基于模型的指标细化与基于案例的指标修订。这些策略源于我们在CaseCAD项目中所发现的关于灵活案例检索需求的认识。[2]

Demex uses two problem reconstruction strategies to provide a better understanding of a design problem: model-based index elaboration and case-based index revision. These strategies evolved from discoveries we made in the CaseCAD project about the need for flexible case retrieval.[2]

通过将当前的问题描述与记忆中的通用知识进行匹配，基于模型的索引细化从记忆中检索出一个或多个模型。这些模型可以从两个方面对设计问题提供更为详尽的描述：首先，从记忆中检索出的各自通常包含比初始问题描述更为丰富的信息（例如，用于描述结构的设计变量数量更多）。设计师可利用这些信息扩展问题的规格集。

Matching an initial problem description against the information in memory, model-based index elaboration retrieves one or more models from memory. The models can provide a better description of the design problem in two ways. First, items retrieved from memory usually contain more information (for example, more design variables to describe the structure). The designer can expand the initial problem specification. The designer can expand the set of specifications with this information.

其次，模型提供与设计相关的可接受值范围。

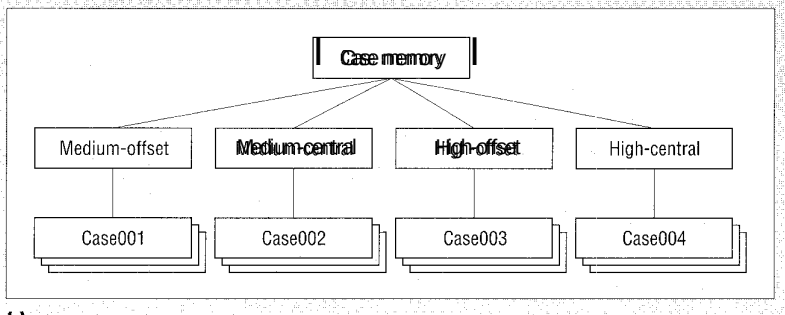
Second, models provide ranges of acceptable values associated with the design

```
**** 中等偏置建筑 ****
FUNCTION
   SUPPORT-BUILDING-TYPE: office, hotel, residential, other
   LATERAL-LOAD-PATH: core, frame, tube, outrigger
   ABOVE-GROUND-LEVELS: 14-30
   ABOVE-GROUND-HEIGHT: 40-120 m
   CORE-LOCATION: east-west, north-south
   CORSYS-LOCATION-CRITICAL-LEVEL: corner, core, edge
   CORSYS-AREA-RATIO-AT-CRITICAL-LEVEL: 0.01-0.5
   CORSYS-LENGTH-RATIO-AT-CRITICAL-LEVEL: 0.1-10
   CRITICAL-WIND-DIRECTION: ew, ns
   CRITICAL-WIND-SPEED: 30-60 m/s
STRUCTURE
   OVERALL-LENGTH: 15-120 m
   OVERALL-WIDTH: 15-120 m
   CORSYS-OVERALL-LENGTH: 2.5-25 m
   CORSYS-OVERALL-WIDTH: 2.5-25 m
   CRITICAL-WIND-AREA: 600-14400 m²
```

(a)

```
案例: INN-ON-THE-PARK
描述: 公园酒店是一座位于悉尼中央商务区的30层国际星级酒店及商业办公楼。该项目提供14层住宿空间，共设有250间四层高的客房，总建筑面积为11,700平方米；5层商业办公空间，建筑面积约为5,000平方米；3层用于酒店配套设施；以及7层地下车库，可容纳170辆汽车。其抗风结构体系由核心筒系统和剪力墙系统组成。
关系实例属于: 中等高度建筑 功能 支撑建筑类型: 酒店 酒店使用面积（m²）：11700 酒店客房数：250 办公使用面积（m²）：5000 地下停车位数：110 行为 结构设计规范：AS1480 抗风规范：AS1170 总层数：30 转换层：5层 地上层数：23 地下层数：7 整体形状：四边形 塔楼形状：四边形 材料：现浇钢筋混凝土 结构整体长度（m）：50 整体宽度（m）：25 标准层面积（m²）：900 地上高度（m）：82 地下高度（m）：20
FUNCTION
   SUPPORT-BUILDING-TYPE: hotel
   STRUCTURAL-CODE: AS1480
   WIND-CODE: AS1170
   TOTAL-LEVELS: 30
   TRANSFER-LEVEL: level-8
   ABOVE-GROUND-LEVELS: 23
   UNDER-GROUND-LEVELS: 7
   OVERALL-SHAPE: quadrangular
   TOWER-SHAPE: quadrangular
STRUCTURE
   MATERIAL: insitu-rc
   OVERALL-LENGTH (M): 50
   OVERALL-WIDTH (M): 25
   STD-AREA (M2): 900
   ABOVE-GROUND-HEIGHT (M): 82
   UNDER-GROUND-HEIGHT (M): 20
```

(b)



(c)

Figure 9. Model and case memory. (a) Design model representation; (b) case representation; (c) memory organization.

图9. 模型与案例记忆。(a) 设计模型表示；(b) 案例表示；(c) 记忆组织。

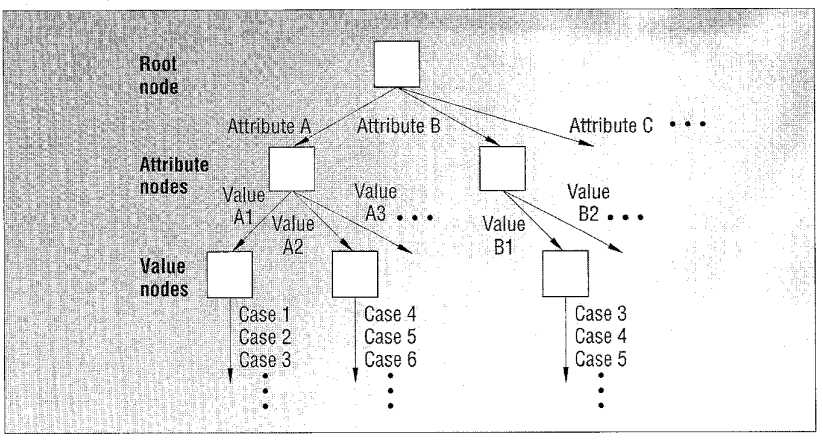


Figure 9. Demex case-indexing tree.

The example in Figure 11, which considers the design of a building's wind-resisting system, illustrates the case-based index revision strategy for problem reformulation. Demex retrieves the case shown in Figure 11b. To achieve wind resistance in structures, the retrieved case uses vertical trusses, not the rigid frames of the original specifications. So, the designer can modify the specifications (see Figure 11c).

In version 2, the attributes `number-` variables that describe the models. The user compares the values in the problem specification and the retrieved model's range of values. Attributes for which the specification value is not in the acceptable range are critical attributes. The user can prune the noncritical parts of a problem specification to focus on the important parts, resulting in a smaller design space.

Figure 10 illustrates model-based index elaboration. The user, interested in designing a medium-height office building, describes the problem to the system (see Figure 10a). Demex uses these problem specifications to retrieve a relevant model. For this example, the best match in the model memory is `medium-rise-office-buildings`, parts of which are shown in Figure 10b.

The retrieved model has more attributes than the user gave in the problem specifications—for example, `location-of-core`, `material`, and `height-to-width-ratio`. If the values are known for any of these attributes, adding them to the problem requirements can improve the focus of case retrieval. Let's assume that Figure 10c shows the user's new problem specifications, with relevant attributes (and their values) added.

A comparison of the problem specifications, version 2, with the design model shows that a value of 2.5 m for the `floor-to-floor-height` attribute is outside the valid range. This identifies a critical attribute that becomes the new problem specifications (see Figure 10d).

The case-based index revision strategy uses cases to modify the problem specifications. Modifications include adding, removing, and altering some or all of the attribute-value pairs in the problem specifications. Altering an attribute-value association could involve modifying the attribute's name, value, or both.

```
Function
Support-building-type:        office
Support-grid-geometry:        rectangular
Structure
Number-of-stories:            30
Overall-width:                23 m
```

(a)

```
Model: medium-rise-office-buildings
Function
Support-building-type:      office
Resist-wind-load:           (1.3 ... 1.7 kPa)
Support-grid-geometry:      (rectangular, circular, ...)
Behavior
Net-area-of-usable-space:   (8000 ... 25000 m2)
Structure
Number-of-stories:          (15 ... 30)
Height-to-width-ratio:      (2 ... 13)
Overall-width:              (20 ... 100 m)
Floor-to-floor-height:      (3 ... 4.5 m)
Material:                   (steel, reinforced-concrete, ...)
Floor-system-type:          (flat-plate, flat-slab, ...)
Wind-system-type:           (core, rigid-frame, ...)
Location-of-core:           (exterior, central, ...)
```

(b)

```
Function
Support-building-type:        office
Support-grid-geometry:        rectangular
Structure
Number-of-stories:            30
Overall-width:                23 m
Floor-to-floor-height:        2.5 m
Location-of-core:             central
```

(c)

```
Structure
Floor-to-floor-height:        2.5 m
```

(d)

Figure 10. Model-based index elaboration in Demex: (a) version 1 of problem specifications; (b) parts of best match retrieved from model memory; (c) version 2 of problem specifications; (d) version 3 of problem specifications.

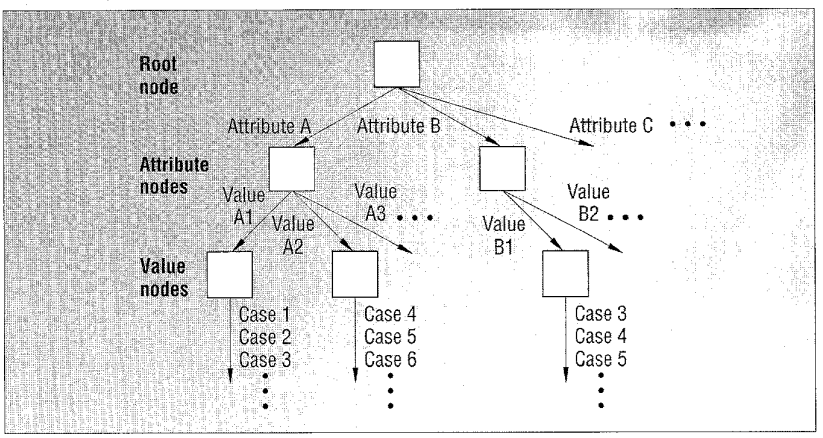


Figure 9. Demex case hierarchy.

图9. Demex案例层次结构。

Figure 9 shows an example of the Demex case hierarchy. Each node in the hierarchy is either an attribute node or a value node. Attribute nodes store the problem-specification attributes used to index the cases. Value nodes store the values of the attributes. Cases are stored in the value nodes under the attribute-value pairs that describe them. For example, Case 1 is stored under the value node for A1 and the value node for B1. Case 2 is stored under the value node for A1 and the value node for A2. Case 3 is stored under the value node for A1 and the value node for B2. Case 4 is stored under the value node for A2 and the value node for A3. Case 5 is stored under the value node for A3 and the value node for B1. Case 6 is stored under the value node for A3 and the value node for B2.

图9展示了Demex案例层次结构的一个例子。层次结构中的每个节点要么是属性节点，要么是值节点。属性节点存储用于索引案例的问题规范属性。值节点存储属性的值。案例存储在描述它们的属性-值对下的值节点中。例如，案例1存储在A1值节点和B1值节点下。案例2存储在A1值节点和A2值节点下。案例3存储在A1值节点和B2值节点下。案例4存储在A2值节点和A3值节点下。案例5存储在A3值节点和B1值节点下。案例6存储在A3值节点和B2值节点下。

Figure 10 illustrates the index refinement process. Figure 10a shows the problem specification the user has entered, describing a medium-height office building. Demex uses these problem specifications to retrieve a relevant model. For this example, the best match in the model memory is `medium-rise-office-buildings`, part of which is shown in Figure 10b.

图10展示了基于模型的索引细化过程。图10a展示了用户输入的问题规范，描述一座中等高度的办公楼。Demex利用这些问题规格从模型库中检索出一个相关模型。在本例中，模型库中的最佳匹配为`medium-rise-office-buildings`，其中的部分如图10b所示。

The retrieved model has additional problem specifications—for example, `location-of-core`, `material`, and `height-to-width-ratio`—that the user didn't enter. If these attributes are important for the design task, the user can add them to the problem specification to improve the focus of case retrieval. Let's assume that Figure 10c shows the user's new problem specifications, with relevant attributes (and their values) added.

检索到的模型具有用户在问题规范中输入的更多属性，例如`location-of-core`、`material`和`height-to-width-ratio`，这些属性是用户没有输入的。如果这些属性对设计任务很重要，用户可以将其添加到问题规范中，以提高案例检索的针对性。假设图10c展示了用户的新问题规格，并添加了相关属性（及其取值）。

Applying the model's constraints to the new problem specifications, version 2, the design model shows that a value of 2.5 m for the `floor-to-floor-height` attribute is outside the valid range. This identifies a critical attribute that becomes the new problem specifications (see Figure 10d).

将问题规范第2版与设计模型进行对比表明，楼层高度属性`floor-to-floor-height`的值2.5米超出有效范围。这确定了一个关键属性，并将其作为新的问题规范（见图10d）。

The case-based index refinement strategy uses case histories to modify problem specifications. Modifications include adding, retrieving, and altering some or all of the attribute-value pairs in the problem specifications. Altering an attribute-value association could involve modifying the attribute's name, value, or both.

基于案例的索引修订策略利用案例来修改问题规范。这些修改包括在问题规范中添加、删除以及更改部分或全部属性-值对。更改某一属性-值关联可能涉及修改该属性的名称、值，或同时修改两者。

```
Function
Support-building-type:        office
Support-grid-geometry:        rectangular
Structure
Number-of-srories:            30
Overall-width:                23 m
```

(a)

```
Model: medium-rise-office-buildings
Function
Support-building-type:        office
Resisx-windload:              (1.33 ... 1.7 kPa)
Support-grid-geometry:        (rectangular, circular, ...)
Behavior
Net-area-of-usable-space:     (8000 ... 25000 m2)
Structure
Number-of-stories:            (15 ... 30)
Beight-to-width-ratio:        (2 ... 13)
Overall-width:                (20 ... 100 m)
Floor-to-floor-height:        (3 ... 4.5 m)
Material:                     (steel, reinforced-concrete, ...)
Floor-system-type:            (flat-plate, flat-slab, ...)
Wind-system-type:             (core, rigid-frame, ...)
Locarion-of-core:             (exterior, central, ...)
```

(b)

```
Function
Support-building-type:        office
Support-grid-geometry:        rectangular
Structure
Number-of-srories:            30
Overall-width:                23 m
Floor-to-floor-height:        2.5 m
Location-of-core:             central
```

(c)

```
Structure
Floor-to-floor-height:        2.5 m
```

(d)

Figure 10. Model-based index refinement: (a) problem specifications, version 1; (b) retrieved model memory; (c) problem specifications, version 2; (d) problem specifications, version 3.

图10. 基于模型的索引细化：(a) 问题规格说明的第1版；(b) 检索到的模型记忆；(c) 问题规格说明的第2版；(d) 问题规格说明的第3版。

of-bays and bay-width in the specification of the rigid frame become number-of-panels and panel-width. In one step, some parts of the description have changed, either in their values or their attributes, while others have remained intact. In this example, no parts of the original specification were dropped (although an alternative way of viewing the altering of an attribute-value association is that it was removed and replaced by an analogous association at the same time).

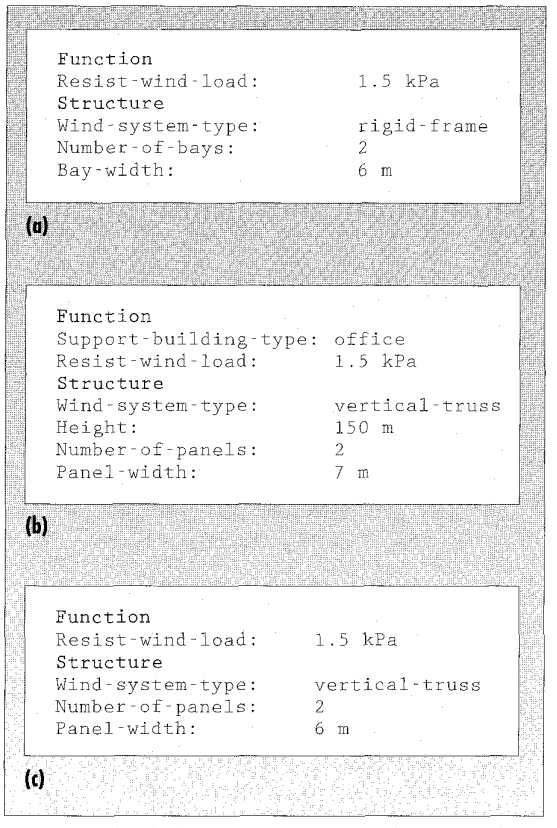


Figure 11. Case-based index revision in Demex: (a) initial problem specifications; (b) retrieved case; (c) second problem specifications.

System options

Our four projects and the experiences we gained from them can themselves serve as case studies on the development of case-based design tools. Although we cannot say that one approach is better than another, their differences point to options in developing CBR design systems.

Representation in case-based design. On the surface, all four systems described here use a similar representation paradigm; however, they have important differences. The similarities are the use of attribute-value pairs for representing a case or subcase and the use of models to support CBR. The attribute-value pair description allows indexing and adaptation of specific cases by means of relatively simple pattern matching based on string comparisons of attributes and values. In addition to the attribute-value pairs, the systems (except CADsyn) document cases with drawings, images, and text descriptions. One way of viewing the attribute-value pairs is to consider them the index to the more comprehensive case description.

The systems do not describe cases in terms of lessons learned but in a formalism relevant to design, such as intended function, actual behavior, and structural attributes. The selection of attributes in a case depends on the other knowledge necessary to support CBR. The attributes in a case description must match those in the model descriptions for the system to recognize that the knowledge in the models is relevant to the case. In CADsyn, the attributes in the case representations also must be the same as those included in the constraints used to identify constraint violations.

The differences among the systems include their case memory organization and the way they use indexes for case retrieval. In CaseCAD and CADsyn, memory is organized as lists of cases and models. All attribute-value pairs are valid indexes, and case retrieval consists of matching all or part of the problem specifications with each case and subcase. This approach is appropriate when case memory contains a limited number of cases and inappropriate when case memory becomes very large.

In Win, the models partition case memory so that the retrieval process first matches the problem specifications with each model. Assuming there are fewer models than cases, this scheme is more efficient than comparing each case. This approach allows only a comparison of the attribute-value pairs in the models; it is valid when the models adequately describe the functional decomposition of relevant cases and there are few models. The use of models to partition case memory limits retrieval to cases that are similar; it does not allow dissimilar but interesting cases to be retrieved.

Demex's tree structure, with attributes and values as branches, is the most efficient representation of a large case memory, because the number of cases does not affect the time needed to find partial matches.

Each system uses models differently. In CaseCAD, the models provide the template for identifying case information during the acquisition of new cases. CADsyn uses models to perform constraint satisfaction. Each model provides the domain space for the attributes in the case description. These attribute domains are the alternative values for fixing violated constraints. In Win, the models partition case memory and provide a heuristic grouping of indexes for problem elaboration and case retrieval. In Demex, the models provide the space from which index elaboration either refines and adds detail to the problem description or identifies critical requirements. Although the specific roles of models differ in each system, the general role of models is to support the reasoning aspects of case-based design.

Reasoning in case-based design. The purpose of reasoning in case-based design can range from design assistance to design automation. CaseCAD, Win, and Demex focus on design assistance through a memory-based system for the recall of similar designs. The reasoning in these systems relies on the user to guide retrieval, modification of problem specifications, and selection of a case to be pursued further. Recording problem specifications and comparing them to cases and models in memory, the reasoner's main task is memory retrieval.

CADsyn's reasoning aims at modeling design synthesis, which is closer to the design automation end of the spectrum. CADsyn's constraint satisfaction approach directly contrasts with CaseCAD's adaptation approach, in which the user changes the case, unsupported by the case-based reasoner. CADsyn is not a fully automated design system, because the user must select the appropriate case or subcase to be adapted and can override the system's selection of the

在刚架的规格中，一个开间和第二个开间的宽度被分别替换为“跨数”和“跨度”。在这一过程中，描述中的某些部分发生了变化，其值或属性均有所调整，而另一些部分则保持不变。在本例中，原始规格中的任何部分均未被删除（尽管另一种看待属性-值关联变更的方式是，这关联被移除，并同时被一个等效的关联所取代）。

specification of the rigid-frame become "number of panels" and "panel width". In one step, some parts of the description have changed, either values or their attributes, while other parts remained intact. In this example, no parts of the original specification were dropped (although an alternative way of viewing the change is that an association was removed and replaced by an analogous association at the same time).

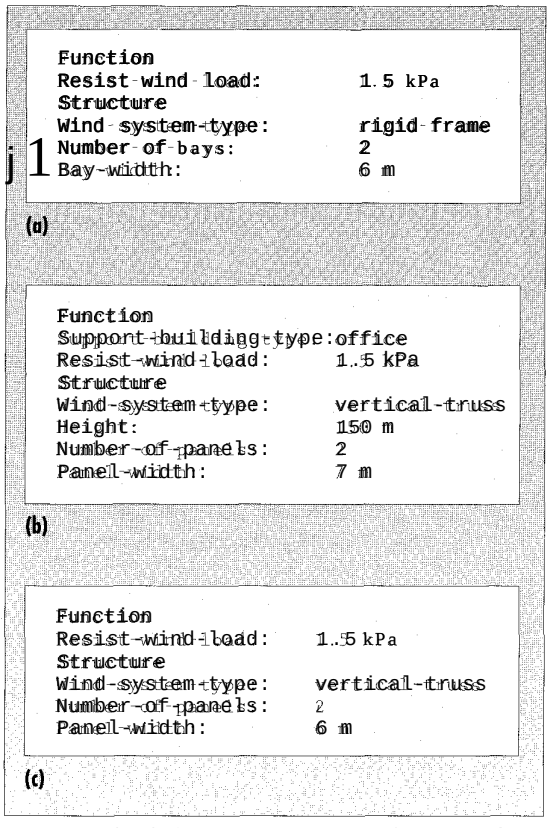


Figure 1. Case-based redesign in Demex: (a) original problem specification; (b) modified case; (c) second problem specification.

图1. Demex中的基于案例的再设计：（a）初始问题规格说明；（b）修改后的案例；（c）第二次问题规格说明。

System options

Our four projects and the experience of others in developing case-based design systems are not sufficient to support rigorous comparisons or case studies on the development of case-based design tools. Although we cannot say that one approach is better than another, their differences point to options in developing CBR design systems.

我们的四个项目及其所积累的经验本身不足以对基于案例的设计工具的开发进行严格的比较或案例研究。虽然我们无法断言某种方法优于另一种，但它们之间的差异为CBR设计系统的开发提供了多种可选路径。

Representation in case-based design. On the surface, all four systems described here use a similar representation: attribute-value pairs. However, there are important differences. The similarities are the use of attribute-value pairs for representing a case or subcase and the use of models to support CBR. The attribute-value pair description allows indexing and adaptation of specific cases by means of relatively simple pattern matching. CaseCAD, CaseWin, and Demex (and other systems not discussed here) use attribute-value pairs to describe cases. CADsyn documents cases with drawings, images, and text descriptions. One way of viewing the attribute-value pairs is to consider them the index to the more comprehensive case description.

基于案例的设计中的表示。表面上看，所有四种系统均采用相似的表示方式：属性-值对。但其间存在重要差异。其相似之处在于：均采用属性-值对来表征案例或子案例，并借助模型支持基于案例的推理。属性-值对的表示方式使得通过基于属性与值的字符串比较进行相对简单的模式匹配，从而实现对特定案例的索引与适应。除CADsyn外，这些系统均以图纸、图像及文本描述的形式对案例加以记录，而CADsyn则以属性-值对的形式对案例加以描述。从某种角度来看，属性-值对可被视为对更为全面的案例描述的索引。

The systems differ in the models they use to support CBR. The models in CaseCAD and CaseWin are structures that describe relevant knowledge for design, such as intended function, actual behavior, and structural attributes. The selection of attributes in a case depends on the other knowledge necessary to support CBR. The attributes in a case description must be consistent with the attributes in the models. For example, in CADsyn, the attributes in the case represen-

这些系统所使用的模型各不相同。在CaseCAD和CaseWin中，模型是描述设计相关知识的结构，例如预期功能、实际行为以及结构属性。案例中属性的选取取决于支持基于案例推理（CBR）所需的其他知识。案例描述中的属性必须与模型中的属性相一致。例如，在CADsyn中，案例中的属性表示能识别出模型中的知识，与该案例相关。

The systems differ in the models they use to support CBR. The models in CaseCAD and CaseWin are structures that describe relevant knowledge for design, such as intended function, actual behavior, and structural attributes. The selection of attributes in a case depends on the other knowledge necessary to support CBR. The attributes in a case description must be consistent with the attributes in the models. For example, in CADsyn, the attributes in the case represen-

这些赋值必须与用于识别约束违规的约束条件中所包含的赋值相一致。

The systems differ in the models they use to support CBR. In CaseCAD and CADsyn, memory is organized as lists of cases and models. All attribute-value pairs are valid indexes, and case retrieval consists of matching all or part of the problem specifications with each case and subcase. This approach is appropriate when case memory is small, but inappropriate when case memory becomes very large.

各系统间的差异主要体现在其案例记忆的组织方式以及用于案例检索的索引机制上。在CaseCAD和CADsyn中，记忆以案例与模型的列表形式组织，所有属性-值对均可作为有效索引，案例检索则通过将问题规格的全部或部分与每个案例及其子案例进行匹配来实现。当案例记忆中的案例数量有限时，这种做法较为适宜；而当案例记忆规模变得庞大时，则显得不够恰当。

In Win, the models partition case memory into several categories, thereby restricting the search for relevant cases. The system first matches the problem specifications with each model. Assuming there are fewer models than cases, this scheme is more efficient than comparing each case. This approach allows only a comparison of the attribute-value pairs in the models. It is valid when the models adequately describe the conditions under which cases are relevant, and when there are few models. The use of models to partition case memory limits retrieval to

在Win系统中，模型通过划分案例库来实现内存管理，使得检索过程首先将问题规格与每个模型进行匹配。假设模型数量远少于案例数量，这种方案相较于逐一比较每个案例更为高效。该方法仅允许对模型中的属性—值对进行比较。当模型能够充分描述相关案例的适用条件，且模型数量较少时，此方法是有效的。利用模型对案例库进行划分，会将检索范围限定于

Demex的树状结构以属性和取值为分支，是六种模型中最高效的表示方式，因为案例数量不会影响查找部分匹配所需的时间。

Demex’s tree structure on attributes and values is the most efficient representation of a large case memory, because the number of cases does not affect the time needed to find partial matches.

每个系统对模型的使用方式各个相同。在CaseCAD中，模型为新案例获取过程中的案例信息识别提供了模板。CADsyn则利用模型进行约束满足；每个模型为案例描述中的属性定义了相应的域空间；这些属性域即为用于修复被违反约束的备选取值。在Win中，模型对案例存储进行划分，并为问题细化与案例检索提供启发式索引。在Demex中，模型则构成了索引细化的搜索空间，既可用于对问题描述进行细化并补充细节，也可用于识别关键需求。尽管各系统中模型的具体作用有所差异，但其总体作用是

The use of models in each system. In CaseCAD, the models provide the template for identifying case information during the acquisition of new cases. CADsyn uses models to perform constraint satisfaction. A model provides the domain ranges for the attributes in the case description. These attribute domains are the alternative values for fixing violated constraints. In Win, the models partition case memory into relatively homogeneous groups of cases. In Demex, the models form the basis for indexing in elaboration and case retrieval. In Demex, the models provide the space from which index elaboration either refines and adds detail to the problem description or identifies key requirements. Although the specific roles of models differ in each system, the general role of models is to support the reasoning aspects of case-based design.

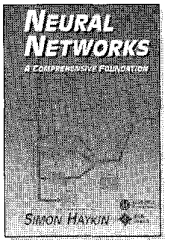
在每个系统中，模型均用于支持基于案例设计的推理过程。

Reasoning in case-based design. The purpose of reasoning in case-based design systems ranges from design assistance to design automation. CaseCAD, Win, and Demex focus on design assistance through a memory-based system for the recall of similar designs. The reasoning in these systems relies on the user to guide retrieval, handle violations, and adapt designs. The user must decide when to modify problem specifications and when to accept a case for adaptation. Recording problem specifications and comparing them to cases and models in memory, the reasoner’s main task is memory retrieval.

基于案例的设计推理。基于案例的设计推理的目的范围可从为辅助设计提供帮助到实现设计自动化。CaseCAD、Win和Demex均通过基于记忆的系统来实现相似设计的检索，从而提供设计辅助。在这些系统中，推理过程依赖于用户引导问题规格的检索与修改，并选择进一步处理的案例。用户必须决定何时对问题规格进行修改，以及何时接受案例以进行适应。记录问题规格说明并将其与记忆中的案例及模型进行比对，推理过程的主要任务是记忆检索。

CADsyn’s reasoning is more ambitious; it uses the models to support constraint satisfaction. CADsyn’s constraint satisfaction approach directly contrasts with CaseCAD’s adaptation approach, in which the user changes the case, unsupported by the case-based reasoner. CADsyn is not a fully automated design system. The user must select the cases to be adapted and can override the system’s selection of the

CADsyn的推理则更为雄心勃勃，它利用模型来支持约束满足。CADsyn的约束满足方法与CaseCAD的适应方法形成鲜明对比，在后者中，用户通过修改案例，而这一操作并未得到基于案例的推理机制的支持。CADsyn并非一个完全自动化的设计系统。因为用户必须选择合适的案例以进行适应，并且可以覆盖系统所作出的选择。



Neural Networks: A Comprehensive Foundation

by Simon Haykin

Represents the most comprehensive treatment available of neural networks from an engineering perspective. This thorough, well-organized, and completely up-to-date book examines all the important aspects of this emerging technology. The text features comprehensive chapter introductions, numerous computer experiments, network problems and solutions, photographs and illustrations and a bibliography and a thorough glossary that will reinforce the key concepts presented in this book.

Contents: Introduction • Learning Process • Correlation Matrix Memory • The Perceptron • Least-Mean-Square Algorithm • Multilayer Perceptrons • Radial Basis Function Networks • Recurrent Networks Rooted in Statistical Physics • Self-Organizing Systems: Hebbian, Competitive Learning, and Maximum Information Preservation • Modular Networks • Temporal Processing • Neurodynamics • VLSI Implementations of Neural Networks • Appendixes • Bibliography and References • Index

720 pages. 1994. Hardcover.
ISBN 0-02-352761-7.
Catalog # BP05687
$55.95 Members / $69.95 List

IEEE COMPUTER SOCIETY

Call toll-free:
1-800-CS-BOOKS
Fax: +1-714-821-4641

invalid system to be modified. CADsyn's reasoning process for design case adaptation, when made explicit, provides alternative design solutions for the designer.

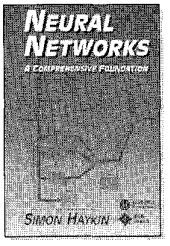
THE FOUR SYSTEMS DESCRIBED here do not represent an exhaustive list of options for implementing case-based design. Although we used similar methods to represent design information and the same design cases, a variety of issues led us to develop different systems. We concentrated on structural design representation so that we could address problem formulation and solution generation. Using CBR for problem formulation led us to consider the role of cases in defining the scope of the problem. Using CBR for generating solutions led us to consider what changes and what stays the same in case adaptation, as well as the knowledge needed to make these decisions and methods of automating the process. The practicality of CBR systems for use by professionals depends on further developments in technology, such as multimedia database management support, and further developments in design knowledge representation.

References

1. M.L. Maher and B. Balachandran, "Multimedia Approach to Case-Based Structural Design," *J. Computing in Civil Engineering*, Vol. 8, No. 3, July 1994, pp. 359–376.
2. M.L. Maher and B. Balachandran, "Flexible Retrieval Strategies for Case-Based Design," in *Artificial Intelligence in Design '94*, J.S. Gero and F. Sudweeks, eds., Kluwer Academic, Dordrecht, The Netherlands, 1994, pp. 163–180.
3. A. Vob et al., "Retrieval of Similar Layouts," in *Artificial Intelligence in Design '94*, J.S. Gero and F. Sudweeks, eds., Kluwer Academic, 1994, pp. 625–640.
4. J.S. Gero, "Design Prototypes: A Knowledge Representation Schema for Design," *AI Magazine*, Vol. 11, No. 4, Winter 1990, pp. 26–36.
5. M.L. Maher and D.M. Zhang, "CADsyn: A Case-Based Design Process Model," *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, Vol. 7, No. 2, 1993, pp. 97–110.
6. M.L. Maher, "Engineering Design Synthesis: A Domain Independent Representation," *Artificial Intelligence in Engineering Analysis and Manufacturing*, Vol. 1, No. 3, 1987, pp. 207–213.
7. R.S. Michalski, "Learning from Observation: Conceptual Clustering," in *Machine Learning: An Artificial Intelligence Approach*, R.S. Michalski, J.G. Carbonell, and T.M. Mitchell, eds., Morgan Kaufmann, San Francisco, 1983, pp. 331–363.
8. T.R. Hinrichs and J.L. Kolodner, "The Role of Adaptation in Case-Based Design," *Proc. DARPA Workshop on Case-Based Reasoning*, Morgan Kaufmann, 1991, pp. 121–132.
9. B. Faltings et al., "Case-Based Representation of Architectural Design Knowledge," *Proc. DARPA Workshop on Case-Based Reasoning*, Morgan Kaufmann, 1991, pp. 307–316.
10. S.H. Wang, *Win: A Case-Based Design Approach to the Structural Design of the Wind Systems of Buildings*, master's thesis, Univ. of Sydney, Australia, 1995.
11. A. Gómez de Silva Garza and M.L. Maher, "Design by Interactive Exploration Using Memory-Based Techniques," to be published in *Knowledge-Based Systems*, Vol. 9, No. 1, 1996.
12. V. Goel and P. Pirolli, "The Structure of Design Problem Spaces," *Cognitive Science*, Vol. 16, 1992, pp. 395–429.

Mary Lou Maher is codirector of the Key Centre of Design Computing and an associate professor in the Department of Architectural and Design Science at the University of Sydney, Australia. Her research interests include case-based reasoning in design, modeling creative design using genetic algorithms, and computer support for collaborative design. She is the editor or coauthor of eight books and over 100 papers on design computing. She received her BS in civil engineering at Columbia University and her MS and PhD in Civil Engineering at Carnegie Mellon University. She can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; phone 61 (2) 351-4108; fax 61 (2) 351-3031; mary@arch su edu au; http://www.arch.su.edu.au/~mary/.

Andrés Gómez de Silva Garza is a PhD student in the Department of Architectural and Design Science at the University of Sydney. His research interests include case-based reasoning, genetic algorithms, and design science. He received his BSc in computer engineering from the National Autonomous University of Mexico in 1991, and his MSc in computer science from the Georgia Institute of Technology in 1993. He is a member of the AAAI. He can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; phone 61 (2) 351-2053; fax 61 (2) 351-3031; andres@arch.su.edu.au.



Neural Networks: A Comprehensive Foundation

by Simon Haykin

This book presents a comprehensive treatment and up-to-date survey of the theory of neural networks from an engineering perspective. The author, a professor at McMaster University, Ontario, Canada, examines all the important aspects of this emerging technology. The book is rich in theory and contains a wealth of computer experiments, problems and solutions, photographs and illustrations, and a useful glossary that will reinforce the key concepts presented in this book.

Contents: • Learning Process • Single Layer Perceptrons • Radial Basis Function Networks • Support Vector Machines • Committee Machines • Principal Components Analysis • Self-Organizing Systems: Hebbian, Competitive Learning, and Maximum Information Preservation • Modular Networks • Temporal Processing • Neurodynamics • VLSI Implementations of Neural Networks • Appendixes • Bibliography and References • Index

720 pages. 1994 Hardcover
ISBN 0-02-352761-7.
Catalog # BP05687 / $69.95 List
$55.95 Members / $69.95 List

IEEE COMPUTER SOCIETY

Call toll-free 800-CS-BOOKS
1-800-CS-BOOKS
Fax: +1-714-821-4641

This paper describes the CADSyn system for case-based design. CADSyn integrates case-based reasoning, constraint propagation, and explicit design knowledge to support the design process. It provides alternative design solutions for the designer.

References

1. M.L. Maher and B. Balachandran, "Multi-Case Design Using a Relational Representation," *Proc. Int'l Conf. Artificial Intelligence in Engineering Analysis and Manufacturing*, Vol. 1, No. 3, 1987, pp. 207–213.

2. M.L. Maher and B. Balachandran, "Flexible Retrieval Strategies for Case-Based Design," in *Artificial Intelligence in Design '94*, J.S. Gero and F. Sudweeks, eds., Kluwer Academic, Dordrecht, The Netherlands, 1994, pp. 163–180.

3. A. Voss et al., "Retrieval of Similar Designs," in *Artificial Intelligence in Design '94*, J.S. Gero and F. Sudweeks, eds., Kluwer Academic, Dordrecht, The Netherlands, 1994, pp. 365–385.

4. J.S. Gero, "Design Prototypes: A Knowledge Representation Schema for Design," *AI Magazine*, Vol. 11, No. 4, 1990, pp. 26–36.

5. M.L. Maher and D.M. Zhang, "CADSyn: A Case-Based Design Process Model," *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, Vol. 7, No. 2, 1993, pp. 97–110.

6. M.L. Maher, "Engineering Design Synthesis: A Domain Independent Representation," *Artificial Intelligence in Engineering Analysis and Manufacturing*, Vol. 1, No. 3, 1987, pp. 207–213.

7. R.S. Michalski, "Knowledge Acquisition through Conceptual Clustering: A Theoretical Framework and Algorithm for Partitioning Data into Conjunctive Concepts," *Int'l J. Policy Analysis and Information Systems*, Vol. 4, No. 3, 1980, pp. 219–244.

8. T.R. Hinrichs and J.L. Kolodner, "The Roles of Adaptation in Case-Based Design," *Proc. 9th Nat'l Conf. Artificial Intelligence*, Morgan Kaufmann, San Mateo, Calif., 1991, pp. 28–33.

9. B. Faltings et al., "Case-Based Reasoning for Structural Configuration of Ribbed Floor Slabs," *Proc. DARPA Workshop on Case-Based Reasoning*, Morgan Kaufmann, 1991, pp. 307–316.

10. S.H. Wang, *A Case-Based Design Approach to the Structural Design of the Wind Systems of Buildings*, master's thesis, Univ. of Sydney, Australia, 1995.

11. A. Gomez de Silva Garza and M.L. Maher, "Design by Interactive Exploration Using Memory-Based Techniques," to be published in *Knowledge-Based Systems*, Vol. 9, No. 1, 1996.

12. V. Goel and P. Pirolli, "The Structure of Design Problem Spaces," *Cognitive Science*, Vol. 16, 1992, pp. 395–429.

Mary Lou Maher is co-director of the Key Centre of Design Computing at the University of Sydney. She is the Professor of Design Computing and Director of the Design Computing Research Laboratory. Her research interests include case-based reasoning in design, modeling creative design processes, and computer support for collaborative design. She is the author of four books and over 100 papers on design computing. She received her BS in civil engineering from Columbia University and her MS and PhD in civil engineering from Carnegie Mellon University. She is a member of the ACM, ASCE, AAAI, and ACADIA. Contact her at the Key Centre of Design Computing, Univ. of Sydney, NSW 2006, Australia; phone 61 (2) 351-4108; fax 61 (2) 351-3031; mary@arch.su.edu.au; http://www.arch.su.edu.au/~mary/.

Andres Gomez de Silva Garza is a PhD student in the Key Centre of Design Computing at the University of Sydney. His research interests include case-based reasoning, genetic algorithms, and computer support for collaborative design. He received his BS in computer engineering from the Instituto Tecnológico Autónomo de México in 1991, and his MSc in computer science from the Georgia Institute of Technology in 1993. He is a student member of the AAAI. He can be contacted at the Dept. of Architectural and Design Science, Univ. of Sydney, NSW 2006, Australia; phone 61 (2) 351-2053; fax 61 (2) 351-3031; andres@arch.su.edu.au.